Syllabus

Detailed Curriculum

Module No.	Unit No.	Contents	No of Hrs.	CO
1	**Introduction to Databases**			
	1.1	Database and Database Users: Introduction, Characteristics of the Database Approach, Actors on the Scene, Workers behind the Scene, Advantages of Using the DBMS approach.		
	1.2	Database System Concepts and Architecture: Data Models, Schemas and Instances, Three-Schema Architecture and Data Independence, Database Languages and Interfaces, Database System Environment, Centralized and Client/server Architectures for DBMSs, Classification of Database Management Systems.	05	CO 1
2	**Conceptual Data Modeling , Database Design Relational Data Model and Database constraints**			
	2.1	Using High-Level Conceptual Data Models for Database Design, Entity Types, Entity Sets, Attributes and Keys, Relationship Types, Relationship Sets, Roles and Structural Constraints, Weak Entity Types, ER Diagrams, Relationship Types of Degree Higher than Two.		
	2.2	The Enhanced Entity- Relationship (EER) Model: Subclasses, Superclasses, and Inheritance, Specialization and Generalization, Constraints and characteristics of Specialization and Generalization Hierarchies, Modeling of UNION types using Categories.	10	CO 2
	2.3	Relational Data Model and Relational database constraints: Relational Model Concepts, Relational Model Constraints and Relational Database Schemas		
	2.4	Relational Database Design by ER and EER-to-Relational Mapping		
3	**Relational Algebra and Structured Query Language (SQL)**			
	3.1	Relational Algebra: Unary Relational operations: SELECT and PROJECT, Relational algebra operations from Set Theory, Binary Relational Operations: JOIN and DIVISION, Additional Relational Operations.		
	3.2	**SQL:** SQL Data Definition and Data Types, Specifying Constraints in SQL, Basic Retrieval Queries in SQL, INSERT, DELETE and UPDATE statements in SQL. More Complex SQL retrieval Queries, Specifying constraints as Assertions and Actions as Triggers, Views in SQL	08	CO 3

Module No.	Unit No.	Contents	No of Hrs.	CO
#Self Learning Security and Authorization using SQL				
4		**Relational Database Design, Indexing, Query Processing and Optimization**	14	CO 4
	4.1	**Relational Database Design:** Basic of Functional Dependencies and Normalization for Relational Databases, Inference rules, Equivalence and Minimal Cover, Properties of Relational Decompositions		
	4.2	**Indexing Structure for files and Physical Database Design:** Types of Single-Level ordered indexes, Multi-level indexes, Dynamic Multi-level indexes using B-Trees and B+ Trees		
	4.3	**Query Processing**: Algorithms for SELECT, PROJECT operations		
	4.4	**Query Optimization**: Query Trees and Heuristics for Query optimization		
#Self-learning – Database performance tuning				
5		**Transaction Processing , Concurrency Control and Recovery**	08	CO 5
	5.1	**Transaction Processing concepts**: Introduction to Transaction processing, Transaction and system concepts, Desirable properties of transactions, Characterizing schedules based on recoverability, Characterizing schedules based on serializability		
	5.2	**Concurrency control techniques**: Two-Phase Lock-based ,Timestamp-based, Multi-version Concurrency Control, Validation-based protocols, Deadlock Handling-Wait for graph		
	5.3	**Recovery Techniques**: Recovery concepts, NO-UNDO/REDO Recovery based on deferred update, Recovery techniques based on immediate update, Shadow paging		

PYQ

wherever necessary

Que. No.	Question Statement	Max. Marks
Q.1	Attempt any **two**	
i)	Describe the three-schema architecture of a database system. Why are mappings between the schema levels necessary	**05**
ii)	State the informal guidelines for relation schema design. Illustrate how violations of these guidelines may be harmful.	**05**
iii)	What is a view in SQL? Explain how views are created and used. Discuss any two advantages of using views in a database system with suitable examples	**05**
Q.2	Attempt any **one**	**10**
i)	Consider the following relational Schema: member(memb no, name) book(isbn, title, authors, publisher) borrowed(memb no, isbn, date) Write SQL queries for the following: a) Find the member number and name of each member who has borrowed at least one book published by "McGraw-Hill". b) Find the member number and name of each member who has borrowed every book published by "McGraw-Hill" c) For each publisher, find the member number and name of each member who has borrowed more than five books from "McGraw-Hill" publisher	
ii)	Construct the B+ Tree for the given input: A PARTS file with Part# as the key field includes records with the following Part# values:20, 11, 14, 25, 30,12, 22, 23, 24	

1/2

SOMAIYA VIDYAVIHAR UNIVERSITY

	Suppose that the search field values are inserted in the give order in a B+ tree with order of internal node m=3 and order of leaf node m_{leaf}=2. Show how the tree will expand and what the final tree will look like	
Q.3	Attempt any **one**	**10**
i)	a) Explain the basic steps of query processing with a suitable diagram. b) Draw a state diagram and discuss the typical states that a transaction goes through during execution.	
ii)	Explain the process of mapping an Entity-Relationship (ER) model to the Relational model. Illustrate with an example.	
Q.4	Attempt the following	**10**
	Consider the three transactions T1, T2, and T3, and the schedules S1 and S2 given below. Draw the serializability (precedence) graphs for S1 and S2, and state whether each schedule is serializable or not. If a schedule is serializable, write down the equivalent serial schedule(s). T1: r1 (X); r1 (Z); w1 (X); T2: r2 (Z); r2 (Y); w2 (Z); w2 (Y); T3: r3 (X); r3 (Y); w3 (Y); S1: r1 (X); r2 (Z); r1 (Z); r3 (X); r3 (Y); w1 (X); w3 (Y);r2 (Y); w2 (Z); w2 (Y); S2: r1 (X); r2 (Z); r3 (X); r1 (Z); r2 (Y); r3 (Y); w1 (X);w2 (Z); w3 (Y); w2 (Y);	
Q.5	Attempt the following	**10**
	Consider the universal relation R={A,B,C,D,E,F,G,H,I,J}and the set following functional dependencies AB->C A->DE B->F F->GH D->IJ What is the key for R? Decompose R into 2NF and then 3NF relations.	

2/3

Module 1.1

Introduction to Databases

Drawbacks in storing data in files

1. Data redundancy and inconsistency
2. Harder to access data
3. Multiple files and formats makes it harder to manage files
4. Hard to add constraints around data or make changes program wise
5. Multiple users accessing same data might corrupt files
6. Hard to keep it secure

Terminologies

Database: A database is an organized collection of related data, stored in such a way that it can be easily accessed, managed, and updated.
Database System: Includes database, DBMS (software) and users along with hardware, os, and programs.
DBMS: A software that helps create, store data in, retrieve data from, update data, delete data on a database. Basically a software that facilitates the management and upkeep of data.

Lifecycle of data:

Data has four stages in its lifecycle

1. Acquisition: The getting of data
2. Aggregation: Combining and organizing collected data.
3. Analysis: Examining data to find patterns.
4. Application: Using insights to make decisions.

Queries: They access data and formulate the result of a request
Transactions: They read data and update some values of it / generate new data altogether and store it in the database.

- A database system is self describing:
    - It has a catalog that stores important information about the database like types, constraints, and data structures
    - This info is called the meta data
    - This allows the DBMS software to understand the database structure and work with multiple applications. For example, the same MySQL program on your pc is able to run a College database and a Bank database simultaneously.

- Insulation between data and programs
    - Called program-data independence
    - This lets you change data structures and organization of a database without changing the access programs. An access program is a script that runs queries and transactions on a database.
- Data abstraction
    - Hides the storage details. We don’t need to mention the block addresses of databases to fetch information. This is done using a data model.
    - You can refer to the data model constructs like table name, column name, etc. without knowing the exact location of these on the drive.
- Multi-user views
    - Helps every user see a different view of the database: you only see columns that concern you
- Multi user transaction processing
    - Allows concurrent users to access and update the database
    - Guarantees that each transaction is either executed or aborted
    - **OLTP:** Allows hundreds of concurrent transactions per second

Advantages

- Low redundancy: Data shared among multiple users
- More integrity: wasteful overlap of resources can be avoided
- No data isolation: information available is current
- No inconsistency
- Higher security: only Database Administration staff access privileged commands
- Allows concurrent use by multiple users
- Optimization of queries: Makes processing efficient
- Backup and recovery
- Represents complex relationships between data
- Drawing inference from data using deductive reasoning.
- Standards can be enforced

Disadvantages

- High initial investment+need for hardware
- DBMS is unnecessary when database and applications are simple and not expected to change
- DBMS is not feasible in an embedded system where a general purpose DBMS might not fit

- DBMS does not suffice when the database cannot handle some complexity in data and when database users need special operations that this DBMS does not provide.

Schemas and instances

Database schema

Database schema is the logical blueprint of a database. It defines tables, data types, and constraints.
Example:
BookID INT PRIMARY KEY
This describes an INT type entity called BookID which is also the primary key of the table.

Schema diagram

An illustration of the database schema

Schema construct

1. Constitutes the elements that define the database schema's structure
2. Represents how data is organized, stored, and interrelated
3. Building blocks of a database schema, including tables, fields, keys, relationships, etc.

In the above BookID example, all words are constructs. BookID is the attribute, INT is the data type, and PRIMARY KEY is the constraint.

Database instance/state

The actual data in a database at any said moment.
Initial database state: Database state at the initial loading time
Valid state: State that satisfies the structure and constraints. Example: if StudentID is PRIMARY KEY and two rows have StudentID = 5, the database state is invalid because the primary key must be unique.

- Schema is also called intension. It changes very rarely.
- State is called extension. It changes every time the database updates.

Schema:

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

State:

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

Database users

1. **Database administrator:** People who use and control databases: maintain and develop database applications
    a. They coordinate all database system related activities
    b. Have a good understanding of the enterprise (whoever the database is for)
    c. They are responsible for defining schemas, structuring the storage, managing authority, and constraining the integrity aspects of the database.
    d. A DBA at an e-commerce company creates user accounts for employees, grants permissions, and schedules daily backups to prevent data loss.
2. **Database designer:** People who design and develop the DBMS Software and related tools.
    a. They identify the data to be stored
    b. They communicate with users and work on feedback
    c. Ensure that the database is optimized and can handle requests efficiently.
    d. Example: A database designer for a hospital management system defines tables for Patients, Doctors, and Appointments, ensuring the appropriate relationships between them.
3. Casual end users:
    a. Access database when needed occasionally
    b. Example: Teacher checking student attendance summary.
4. Naive end users:

    a. They access the database using the GUI
    b. Example: Social media users post and read information from websites
5. Sophisticated end users:
    a. Use advanced tools to access database or write queries.
    b. They work closely with the stored database
    c. Example: Engineers
6. Standalone end users
    a. Example: A researcher uses MS Access for managing survey data.
    b. Maintain personal databases to access information quickly.

Data independence

The ability of the data to change the schema at one level of the database without having to change the schema at the next higher level

Physical Data Independence

- The ability to change the physical level without affecting the logical or Conceptual level.
- Physical data independence gives us the freedom to modify the - Storage device, File structure, location of the database, etc. without changing the definition of conceptual or view level.
- Example: DB admin changes file organization or adds an index to make queries faster. The tables and programs using the database remain the same. That ability is physical data independence.
- Changing the storage devices like SSD, hard disk and magnetic tapes, etc.

Logical Data Independence

- The ability to change the logic behind the logical level without affecting the other layers of the database
- Allows us to make changes in a conceptual structure like adding, modifying, or deleting an attribute in the database.
- If there is a database of a banking system and we want to add the details of a new customer or we want to update or delete the data of a customer at the logical level data will be changed but it will not affect the Physical level or structure of the database.

Data abstraction

Definition: Extracting the necessary data by ignoring the remaining irrelevant details.

Three levels

1. Physical
    a. Lowest level of abstraction
    b. Indicates how data is stored
    c. Used to describe the whole database architecture
2. conceptual/logical
    a. We can describe the database logically, like tables and relationships, without describing how the data is physically stored on disk
    b. Link between external and internal schema
3. External or view level
    a. Highest level
    b. Describes the user interaction
    c. Provides a GUI to the user using which they interact with the database.

DBMS architecture

Three types

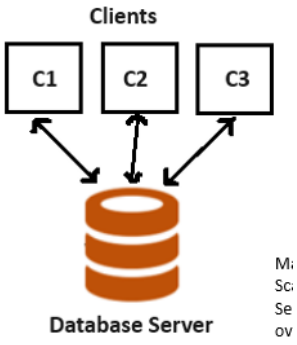
1. 1-tier
2. 2-tier
3. 3-tier

1-tier Architecture

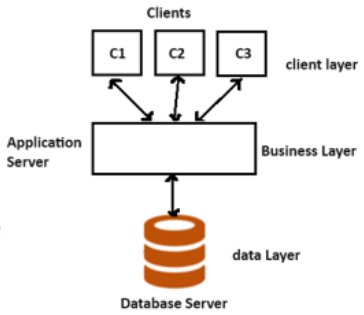
- The database is directly available to the user.
- Any changes made will directly reflect on the database
- Used for developing local applications where programmers are able to directly communicate with the database and get quick responses

2-tier architecture

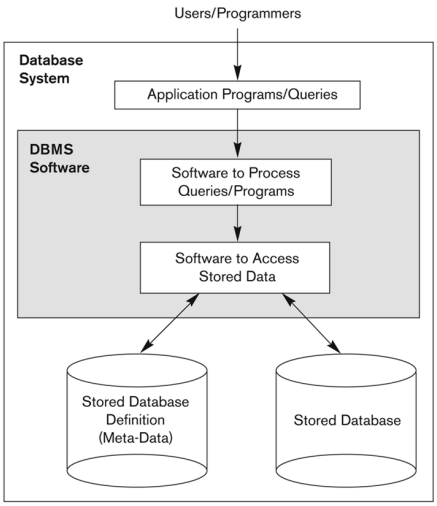
- Similar to a client-server model
- Applications on client can communicate with database on server side. This is done using APIs
- The user interface is run on client side
- The server side is responsible for database and backend
- To communicate with the DBMS, client side establishes a connection with server side.



3-Tier architecture

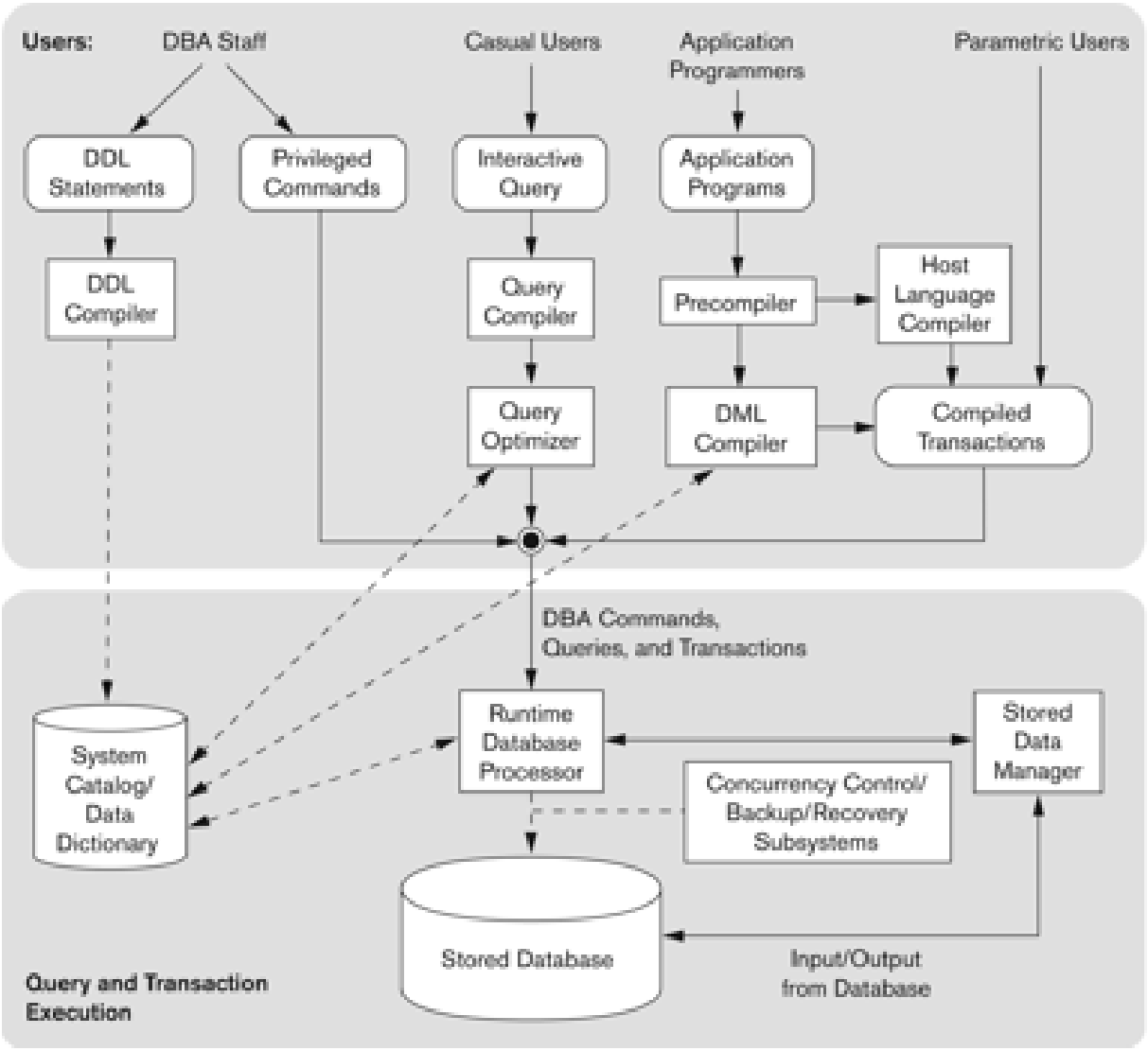


- Clients cannot directly communicate with server.
- Application on client interacts with an application server. This application server in turn talks to the database system.
- Used in case of large web applications.
- Defines schemas at three levels:
    - Internal schema: describes physical storage structures and access paths
    - Conceptual schema: Describes the structure and constraints for the database
    - External Schema: Describes the user views.



Mapping: It is used to transform request and response between various database levels of architecture

- Not good for small databases as it takes higher time
- In external/conceptual mapping: it is important to transform request from external to conceptual
- In conceptual/internal: DBMS can transform request from conceptual to internal level.



Dbms components

Storage manager: manages how data is stored on disk. It handles storing, retrieving, and updating data in physical storage.
Buffer manager: Caches the data on priority and manages memory.
File manager: Disk space management and data structure allocation
Authorization and integrity manager
Transaction manager

Module 1.2

Database System Concepts and Architecture

Data model

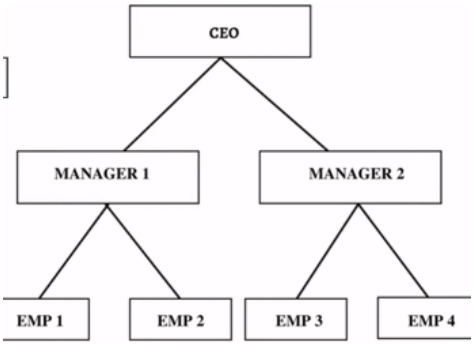
1. A data model is a set of concepts used to describe the structure of a database, the operations that can manipulate the data, and the constraints the data must follow.
2. It is an abstract model that organizes data elements in some format.
3. Constructs are used to define the database structure
    a. They include elements, their data types, and group of elements and their relationships.
    b. Constraints specify restrictions on valid data.
4. Data model operations
    a. Used to specify database retrievals and updates
    b. Operations on the data model may include basic model operations (e.g. generic insert, delete, update) and user-defined operations (e.g. compute_student_gpa, update_inventory)

Categories

1. **Conceptual data models:** Provide concepts close to real world data
    a. Called object based data models
    b. Example: Entity Relationship (ER) model.
2. **Physical data models:** Provide concepts that describe how data is stored in a computer
    a. Example structures: B+ tree index, hashing, record storage.
3. **Implementation data models:** Describe the database in a way close to how it is implemented in DBMS, but still hide low level storage details. It describes tables and relationships.
   Example: Tables: Student(id, name). Course(id, title).
   Enroll(student_id, course_id).

History of Data Models

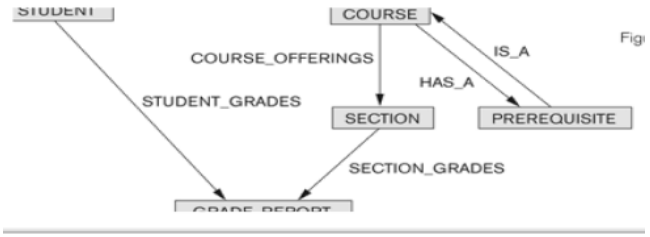
Hierarchical Data Model



- Data is organized in a **tree-like** structure
  Each record is one parent with many children
- One to many relationships between two different data kinds
- Advantages:

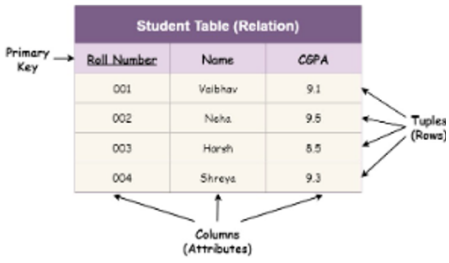
    - Simple operation and construction
    - Simple language like GET, GET UNIQUE
    - This model fits the real world hierarchical data like in an organization
- Disadvantages
    - Little scope for query optimization
    - Data visualisation is linear: Data is accessed by moving from one record to next, reducing speed
    - Navigational and procedural nature of processing: user must follow fixed path to access data

Network Model



- Many to many relationship
- Data is arranged in a **graph**, a child can have multiple parents
- Parent nodes -> owners
  Child nodes -> members
- Advantages
    - Models complex relationships and represents the semantics of add and delete
    - Data is retrieved faster
    - Data can be accessed in multiple ways
    - Navigational language: uses constructs like FIND, FIND NEXT within set, etc
- Disadvantages
    - As the number of relationships increases, system becomes very complicated
    - User has to have a very firm understanding of the model
    - Alteration like update, delete, etc, is very difficult

Relational Model



- Data is organized in 2-d tables
- Relationship is maintained by storing a common field
- Currently the most dominant model

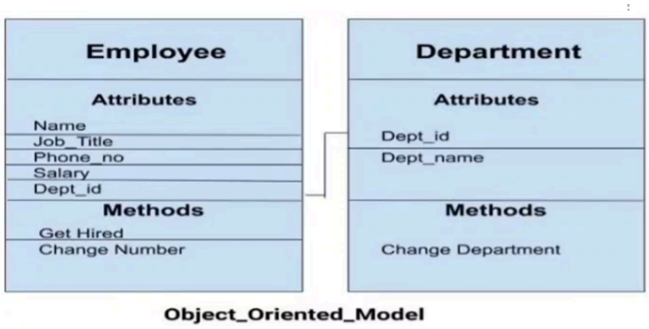
ER Model

- Logical representation of data as objects and their relationships
- Objects are called entities
  Relationship is an association among entities
- Widely used in database design

- Set of attributes describe the entities

Object Oriented DM

- Multiple objects are connected together
- Designed to integrate OOP paradigms with database management



Object_Oriented_Model

Object Relational Models

- Refers to a combination of relational DB model and object oriented database model
- Supports classes, object, inheritance etc found in OOP and also data types, tabular structures, etc found in relational.

DBMS Languages

Data Definition Language DDL

- Language to define database structure
- Defines internal and external schemas
- In some DBMSs, storage definition language and view definition language are used to define internal and external schema.
- Examples: CREATE, ALTER, DROP, TRUNCATE.

Data Manipulation Language DML

- Language to insert/update/delete data
- Examples: INSERT, UPDATE, DELETE, SELECT.

High level language: user writes what needed like in SQL. also called declarative language. Specifies what to do, not on how to do it.
Low level language: user writes step by step procedure in a programming language. Loop constructs and positioning pointers are used to retrieve info.

DBMS Interfaces

- Stand-alone query interface: user directly writes SQL queries in DBMS console.
- Programmer interface
    - DML commands are written inside programs like C, Java.
    - Procedure call approach: program calls the predefined database procedures to access or modify data.

    - Database programming language approach: A programming language based on SQL to write commands.
- User friendly interface: easy interfaces like menus, forms, or graphics to access database without writing SQL. Or Natural language interface: Requests are written in english
- Speech as input/output
- Web browser interface
- Parametric interfaces: let users run predefined queries by entering values (parameters).

Tools

Data dictionary

Data dictionary means a storage of database information. It stores schema, program details, user info, rules.

- Active data dictionaries are automatically used by DBMS. Example: DBMS automatically checking table schema before executing SQL.
- Passive data dictionaries are only read by users or DBA. Example: document or catalog where DBA reads database structure information.

ADE

Application development environments and CASE tools help programmers design and build database applications easily.

Classification of DBMS

Based on Datamodel

Traditional: Relational, Network, Hierarchical.
Emerging: Object-oriented, Object-relational

Based on User

Single-user (typically used with personal computers)
vs. multi-user (most DBMSs).

Based on location

Centralized (uses a single computer with one database)
vs. distributed (uses multiple computers, multiple databases)

Distributed DBMS: DDBMS

Distributed DBMS means database spread across many computers.

1. Homogeneous DDBMS: all databases use same DBMS
2. Heterogeneous DDBMS: databases use different DBMS.

A multidatabase system connects many independent databases. Known as client-server based database because they support a set of database servers supporting a set of clients.

Cost considerations

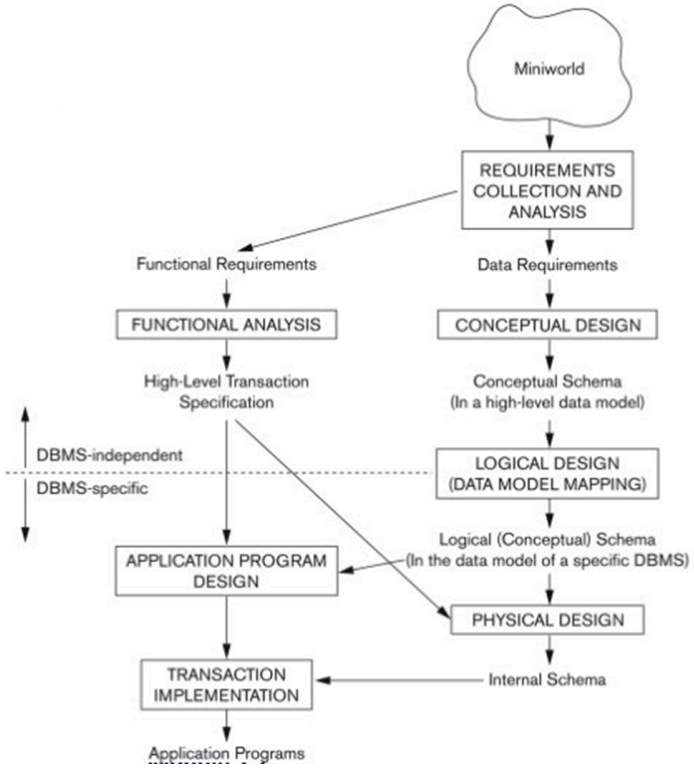
- Ranges from free to millions of dollars
- Free: mySQL and PostgreSQL
- Commercial DBMS offer specialized modules that offer additional functionality when purchased. A cartridge is a special add-on module in a DBMS that adds extra features, like text search or spatial data support.
- Different licenses

Module 2.1

Data Modeling

Database design process

1. The process starts with understanding the miniworld (real-world system).
2. In requirements collection and analysis, both functional and data needs are identified.
3. Functional requirements lead to functional analysis and transaction specifications.
4. Data requirements lead to conceptual design, then logical design (DBMS-specific schema), followed by physical design.
5. Finally, application programs and transactions are implemented to operate on the database.



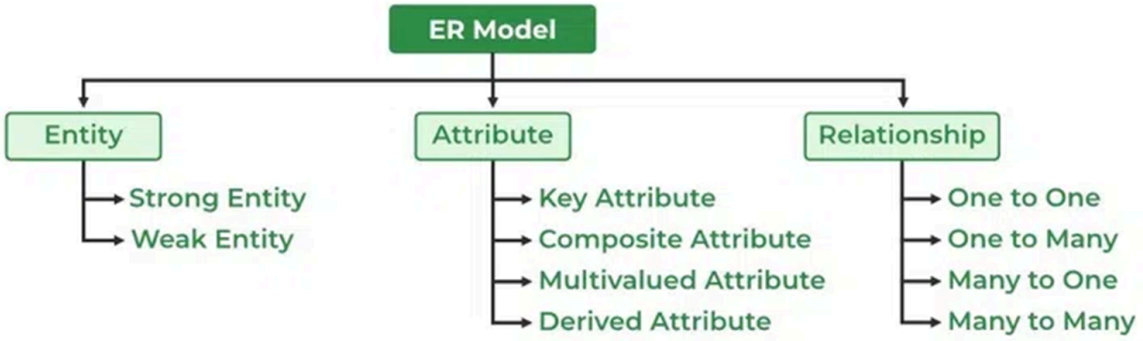
ER model: Entity Relational Model

Used for identifying entities that need to be represented in the database, and for identifying the relation between entities.
The ER model shows the overall structure of a database using diagrams.

Why use ER diagrams

1. Represents the ER model, making it easier to convert to tables
2. Provide the purpose of real world modeling of objects
3. Require no technical knowledge
4. Very easy to understand and create
5. Standard solution for data visualization.

Components



Entity example: Student, course
Attribute example: Student_ID, Name, Age.
Relationship: Student enrolls Course.

Entity: An object with physical existence
Entity set: Set of many entities.
Attribute: Any property that can be associated with an entity
Relationship: How two entities could relate to each other

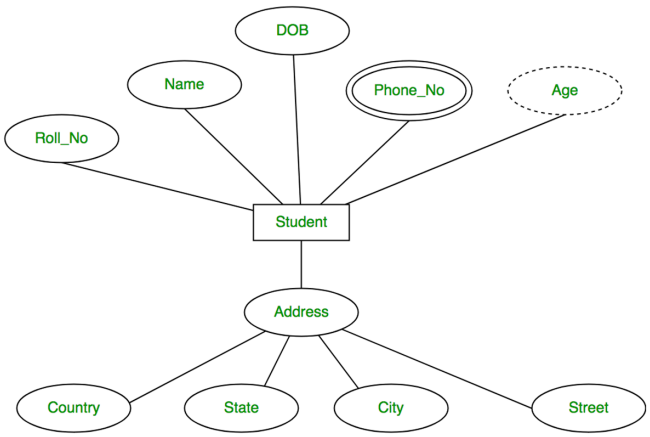
Entity

1. Strong entity
    a. Has a key attribute
    b. Does not depend on another entity in the schema
    c. Has a primary key
2. Weak entity
    a. Need another entity's key to identify
    b. Example: Employee is strong entity. Dependent is weak entity. Dependent can be employee's child or spouse. It exists only if employee exists.

c. Example: apartment is a strong entity and a room in it is a weak entity.

Attributes

1. Key attribute
   a. Attribute that uniquely identifies each entity, for example the primary key
2. Composite attribute
   a. An attribute composed of many other attributes
   b. Example: address includes street, city, state, etc
   c. Oval extended by many ovals
3. Multivalued attribute
   a. Consists more than one value for an entity: example email id or phone number
   b. Is shown by a double oval
4. Derived attribute
   a. An attribute that is calculated from another attribute
      Example: bonus can be calculated from salary/age from DOB
   b. Shown by dotted oval



Relationship

1. Relates two or more entities with some meaning
2. Same type relationships are grouped into a relationship type
3. The degree of a relationship type is the number of participating entities
   Degree 3 means three entities involved. Example. Doctor prescribes Medicine to Patient.

Types:

a. Binary

Relationship between two entities
Student enrollsIn course

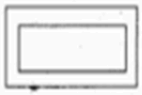
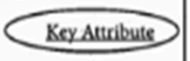
b. Ternary
   Relationship between three: Doctor prescribes medicine to patient.
c. N-ary
   relationship between more than two entities
d. Recursive
   A relationship between two entities of similar entity type
   Example: An employee works for Manager and Manager manages employee.

Participation constraint: This is applied on the entity participating in the entity set. Each entity in the set must participate in a relationship.

Total participation is shown by double line. Total participation means every entity must take part in the relationship. For example: All students must enroll in a course.

Partial participation is shown by a single line. It means some entities may not take part. For example, some students might not join a club.

Symbols in an ER model

ER Component	Description (how it is represented)	Notation
Entity – Strong	Simple rectangular box	
Entity – Weak	Double rectangular boxes	
Relationships	Rhombus symbol - Strong	
between Entities	Rhombus within rhombus – Weak	
Attributes	Ellipse Symbol connected to the entity	
Key Attribute for Entity	Underline the attribute name inside Ellipse	
Derived Attribute for	Dotted ellipse inside main ellipse Entity	
Multivalued Attribute	Double Ellipse for Entity	

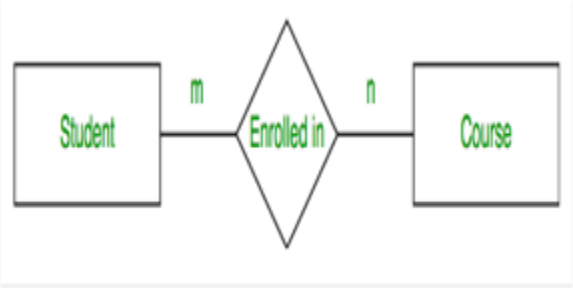
Mapping Constraints

Also called cardinality ratios
It expresses the number of entities to which another entity can be associated with. For a binary relationship, the mapping cardinality has to be among the following:

1. One to one
    a. Entity in A is associated with at most one entity in B and vice versa.
       Example, Roll number and student
    b. We draw a directed line to indicate this
2. One to many
    a. Entity in A is associated with any number of entities in B(zero or more)
    b. An entity in B can be associated with at most one entity in A

    c. A proctor can have multiple students but one student can only have one proctor.
3. Many to one
    a. An entity in A is associated with at most one entity in B
    b. An entity in B can have multiple associations in A
    c. Example: students and department
4. Many to many
    a. An entity in A is associated with any number of entitis in B and vice versa.
    b. Example: Subjects and students

When writing these relationships, a number on top of the line denotes the number of entities in A relating to B and so on.
Example
This is known as cardinality. Definition: Cardinality means how many entities of one set can relate to another.



For many to one and one to many, we follow m=0/1 and n=0/1

Types of Data Models

Hierarchical

Data is organized in a tree structure
Each record consists of one parent and many children

Network

Any record can have several parents
Represented in a graph

Flat Data

Database is represented in rows and columns
All data is kept in a single plane
Cannot store large amounts of data
Semistructure: does not have tabular structure

Associative

Data is separated in two sections
Entity and relationship sections
Items and links exist

Context Data

Consists of more than one data model.
Different types of data models include:
Conceptual – high level, ER diagrams
Logical – tables, relations (relational model)
Physical – how data is actually stored on disk

Module 2.2, 2.3

Enhanced Entity-Relationship (EER) Modeling

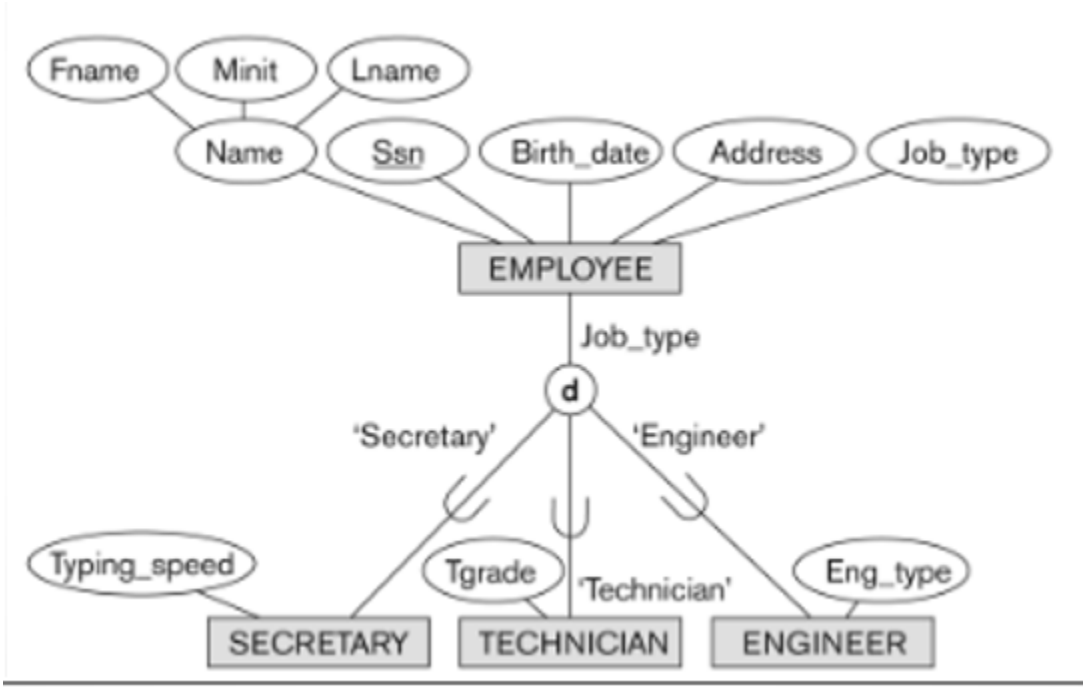
EER model is a diagrammatic technique that displays

- Superclass: general entity type. Example Person.
- Subclass: specialized type of a superclass. Example Student from Person
- Specialization: dividing one entity into smaller types. Example Person → Student, Teacher.
- Generalization: combining entities into one. Example Student, Teacher → Person.
- Union (category): subclass from multiple superclasses. Example Owner from Person and Company.
- Aggregation: treating relationship as entity. Example Employee works_on Project, here works_on is treated as entity.

Subclass and superclass

These are also called IS-A relationships

Here, Employee is a superclass of Secretary, Technician, and Engineer. The circle with "d" means disjoint specialization. An employee can be only one of these:



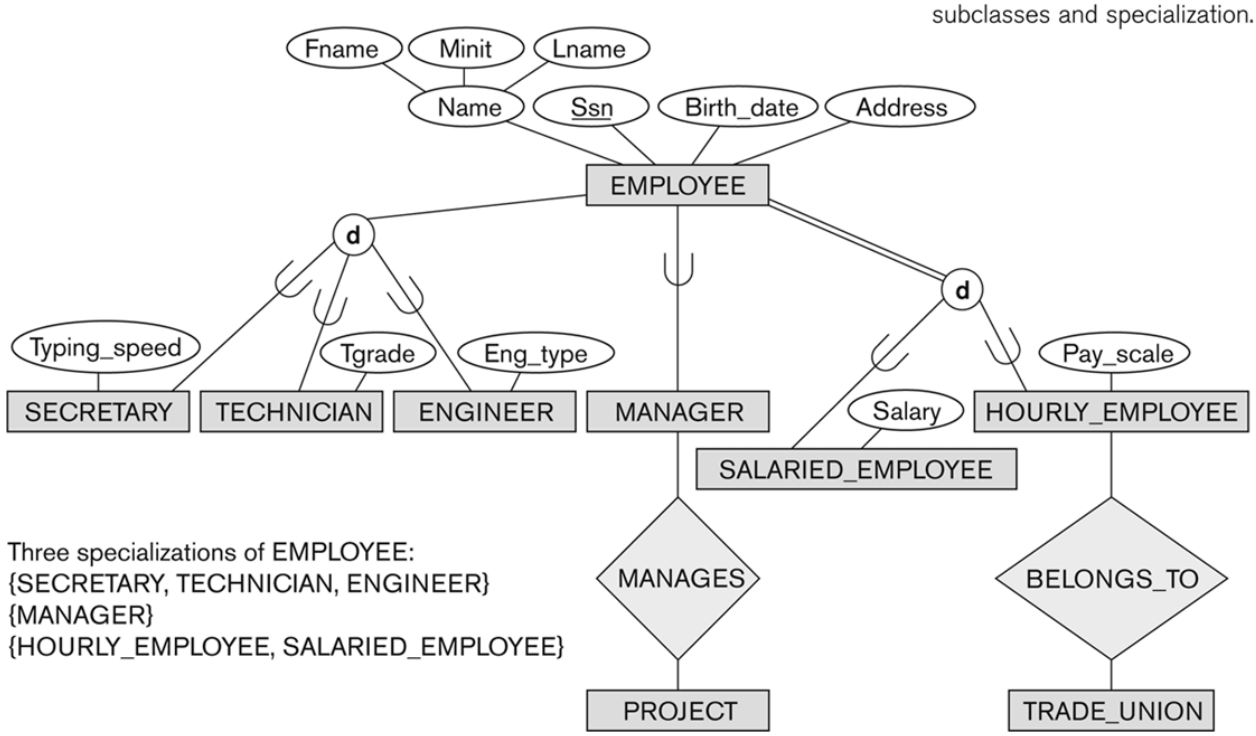
A subclass entity inherits all attributes of the superclass entity, and all relationships.

Above, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, …, from EMPLOYEE

Specialization

Process of defining a set of subclasses of a superclass.
Example: {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon job type.
Attributes of a subclass = specific / local attributes

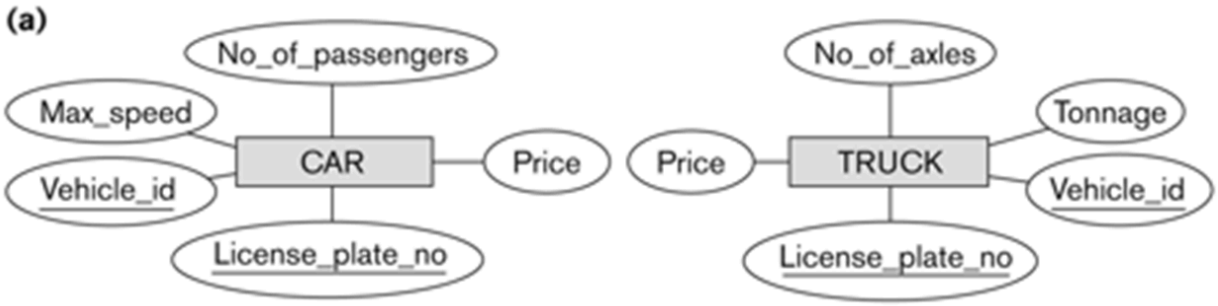
Figure 4.1
EER diagram notation to represent subclasses and specialization.



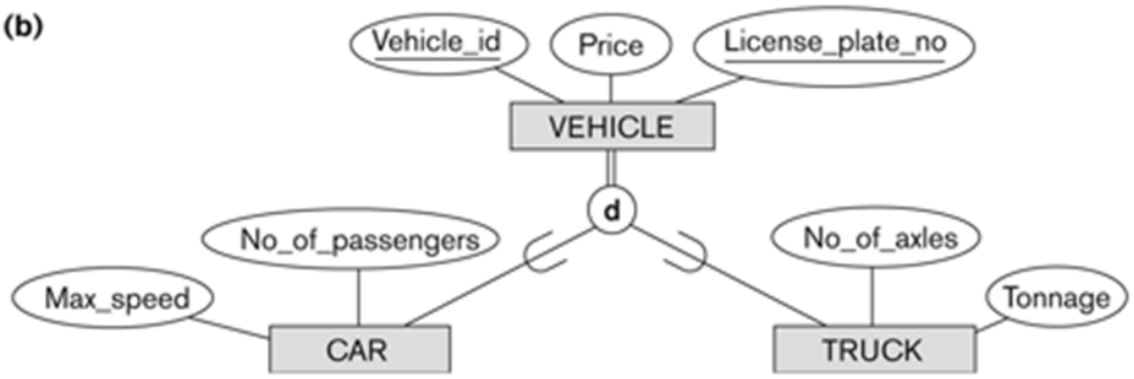
Notice how there is no d mark for manager: An employee can be a manager and an engineer/secretary/technician.

Generalization

Reverse of specialization
Several classes with some common features are generalized into a superclass
Original classes become its subclasses
Example: car and truck -> vehicle
So this:



Becomes this:



Attribute-defined specialization: subclasses decided by value of one attribute. Example JobType decides Secretary, Technician, Engineer. No human intervention is needed to decide the membership.

Defining Attribute (A): A single attribute of the superclass G whose value determines the membership of an entity in a particular subclass
Predicate: A condition or rule used to define membership in each subclass, where ci is a constant value from the domain of the attribute A.
If JobType = "Engineer" → entity goes to Engineer subclass.

User-defined specialization: subclasses decided by user choice. Example user marks an Employee as Manager. The administrator decides who belongs to which subclass.

Constraints

1. Disjointness
   a. Specified by d: elements are disjoint and can belong to just one specialization
   b. Specified by o: Same entity can be part of more than one subclass
2. Completeness
   a. Total: Every entity in superclass must be a member of some subclass

Double line
b. Partial
Not every entity needs to belong to subclass.
Single line
Example Employee → Manager, Engineer

Total four types of specialization/generalization

1. Disjoint, total: Employee -> Permanent, Hourly
2. Disjoint, partial: Employee → Manager, Engineer
3. Overlapping, total: Person → Student, Employee
4. Overlapping, partial: Person -> Doctor, Researcher

Hierarchy: subclass has only one superclass. Example Vehicle → Car → ElectricCar.

Lattice: subclass can have many superclasses. Example TeachingAssistant from Student and Employee.

Single inheritance: subclass inherits from one superclass.

Multiple inheritance: subclass inherits from many superclasses.

Inheritance: subclass gets attributes of all superclasses.

Shared subclass: subclass with more than one superclass. Example TeachingAssistant.

Category

Category (union subclass): subclass formed from different superclasses. Example Owner from Person, Bank, Company. A category member must exist in at least one superclass.

Difference: shared subclass is intersection. Category is union. When the specialization is neither disjoint nor overlapping—it’s a union.

Class

Class: any entity type or subclass. Example Person.
Subclass: smaller class inside another class. Example Student inside Person. $S \subseteq C$ means every Student is also a Person.
Superclass: the larger class. Example Person is superclass of Student.
Subclass inherits attributes and relationships of superclass. Example Student inherits Name, Age from Person.

Module 2.4

ER to Relational Mapping

Mapping of strong entity

1. For each strong entity in schema, create a relation R that includes all the simple attributes of E.
2. Choose one of the key attributes of E as the primary key for R.
3. If the chosen key of E is composite, the set of simple attributes that form it will together form the primary key of R.

Mapping of weak entity

1. Weak entity W becomes a new table R.
2. Add all attributes of W to R.
3. Add primary key of owner entity E as foreign key in R.
4. Primary key of R = owner primary key + weak entity partial key.

Mapping of binary 1:1 relations

1. For each binary relationship, identify the relations S and T that correspond to entity types in R
2. Foreign key approach: Include a foreign key in one relation
3. Merged relation: Merged relation means two entities in a 1:1 relationship can be combined into one table. Used when both entities always participate in the relationship.
4. Cross reference: Cross-reference relation means creating a separate table to store the relationship between two entity tables. This table stores the primary keys of both entities.

Mapping of binary 1:N relations

1. Find the entity on the N side.
2. Add the primary key of the 1 side as a foreign key in the N side table.
3. Also add attributes of the relationship to the N side table.

Mapping of binary M:N relations

1. Create a new table for the relationship.
2. Add primary keys of both entities as foreign keys.
3. These two keys together become the primary key.

4. Also add attributes of the relationship.
5. Example.
   WORKS_ON table: ESSN, PNO, HOURS.
   Primary key: ESSN + PNO.

Multivalued Attribute

1. Create a new table.
2. Add the multivalued attribute.
3. Add the primary key of the entity as foreign key.
4. Primary key = entity key + multivalued attribute.
5. Example.
   DEPT_LOCATIONS table: DNUMBER, DLOCATION.
   Primary key: DNUMBER + DLOCATION.

N-ary relationship

1. Create a new table for the relationship.
2. Add primary keys of all participating entities as foreign keys.
3. Also add attributes of the relationship.
4. Example.
   SUPPLY table: SNAME, PARTNO, PROJNAME.
   Primary key: SNAME + PARTNO + PROJNAME.

Mapping specialization/generalization

1. Option 8A: create one table for superclass and separate tables for each subclass.

   ALL

2. Option 8B: create tables only for subclasses. Superclass table not created.

   EVERY ENTITY IN SUBPERCLASS MUST BELONG TO AT LEAST ONE SUBCLASS

3. Option 8C: create one table for all. Use one type attribute to show subclass.

   DISJOINT

4. Option 8D: create one table for all. Use many type attributes, one for each subclass.

OVERLAPPING

Mapping of Union Types (Categories)

1. Union type mapping means creating a new table for the category. Example: OWNER can be a Person OR a Bank OR a Company
2. Problem: Person has SSN, Bank has BankID, Company has CompanyID, all different keys. Because superclasses have different keys, a new artificial key is created. This new key is called a surrogate key.
3. Example: OWNER table with OwnerId as primary key.

Module 3.1

Relational Algebra

Unary Relational Operations

SELECT (symbol: σ (sigma))

The selection condition acts as a filter
Select the EMPLOYEE tuples whose department number is 4: σ DNO = 4 (EMPLOYEE)
commutative

PROJECT (symbol: π (pi))

The list of specified columns (attributes) is kept in each tuple The other attributes in each tuple are discarded
To list each employee's first and last name and salary, the following is used: π LNAME, FNAME,SALARY(EMPLOYEE)
If EMPLOYEE had DOB column. It would be discarded.
Not commutative

RENAME (symbol: ρ (rho))

- ρS (B1, B2, …, Bn)(R) changes both: the relation name to S, and the column (attribute) names to B1, B1, …..Bn
- ρS(R) changes: the relation name only to S
- ρ(B1, B2, …, Bn)(R) changes: the column (attribute) names only to B1, B1, …..Bn
- Number of columns should always be = n

Relational Algebra Operations From Set Theory

UNION (∪)

The result of R ∪ S, is a relation that includes all tuples that are either in R or in S or in both R and S
Duplicate tuples are eliminated
R and S must have same number of attributes. Each pair of corresponding attributes must be type compatible (have same or compatible domains)

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

To retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or directly supervise an employee who works in department 5 (RESULT2 below)
We can use the UNION operation as follows:
DEP5_EMPS ← σDNO=5 (EMPLOYEE)
RESULT1 ← π SSN(DEP5_EMPS)
RESULT2 ← πSUPERSSN(DEP5_EMPS)
RESULT ← RESULT1 ∪ RESULT2

INTERSECTION (∩)

The result of the operation R ∩ S, is a relation that includes all tuples that are in both R and S
The two operand relations R and S must be “type compatible”
DEP5_EMPS ← σ DNO=5 (EMPLOYEE)
DEP5_SUPERVISORS ← π SUPERSSN (DEP5_EMPS)
DEP5_EMPLOYEE_SSNS ← π SSN (DEP5_EMPS)
RESULT ← DEP5_EMPLOYEE_SSNS ∩ DEP5_SUPERVISORS

DIFFERENCE (or MINUS, –)

The result of R – S, is a relation that includes all tuples that are in R but not in S
The two operand relations R and S must be “type compatible”
DEP5_EMPLOYEE_SSNS ← π SSN (DEP5_EMPS)
DEP5_SUPERVISORS ← π SUPERSSN (DEP5_EMPS)
RESULT ← DEP5_EMPLOYEE_SSNS − DEP5_SUPERVISORS

CARTESIAN PRODUCT (x)

CARTESIAN (or CROSS) PRODUCT Operation
This operation is used to combine tuples from two relations in a combinatorial fashion.
The two operands do NOT have to be "type compatible”
RESULT ← σ DNO=5 (EMPLOYEE × DEPARTMENT)
har ek row ko doosri table ki har ek row se combine karo.

Binary Relational Operations

JOIN

JOIN combines two relations. Uses related attribute values. Produces tuples containing attributes from both relations.

The sequence of CARTESIAN PRODUCT followed by SELECT is used quite commonly to identify and select related tuples from two relations.
RESULT ← σ EMPLOYEE.DNO = DEPARTMENT.DNO (EMPLOYEE × DEPARTMENT)

Suppose that we want to retrieve the name of the manager of each department. To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
We do this by using the join >< operation.
DEPT_MGR ← DEPARTMENT ⋈ MGRSSN=SSN EMPLOYEE

DIVISION

DIVISION finds tuples related to all tuples. Used for "for all" queries.
Example: students who took all courses.
R1 ÷ R2 = tuples of R1 associated with all tuples of R2.
RESULT ← WORKS_ON ÷ PROJECTS
Meaning: Give me only those employees who have worked on ALL the projects listed in the PROJECTS table

Complete set of relational operators

The set of operations including SELECT σ, PROJECT π , UNION ∪, DIFFERENCE − , RENAME ρ, and CARTESIAN PRODUCT X is called a complete set because any other relational algebra expression can be expressed by a combination of these five operations.

Summary

Table 6.1
Operations of Relational Algebra

Operation	Purpose	Notation
SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{<\text{selection condition}>}(R)$
PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{<\text{attribute list}>}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{<\text{join condition}>} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{<\text{join condition}>} R_2$, OR $R_1 \bowtie_{(<\text{join attributes 1}>), (<\text{join attributes 2}>)} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{<\text{join condition}>} R_2$, OR $R_1 *_{(<\text{join attributes 1}>), (<\text{join attributes 2}>)} R_2$ OR $R_1 * R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Additional Relational Operations

OUTER JOINS:

Outer join keeps unmatched tuples. Missing values filled with NULL.

- **Left Outer Join (⟕)**: Includes all tuples from the left relation.
- **Right Outer Join (⟖)**: Includes all tuples from the right relation.
- **Full Outer Join (⟗)**: Includes all tuples from both relations.

OUTER UNION

AGGREGATE FUNCTIONS

These compute summary of information: for example, SUM, COUNT, AVG, MIN, MAX

- Notation:
  Often uses a script '$\mathcal{F}$' (ʃ) or '∑' (sigma) symbol, followed by the function and attribute, and the relation name.
- Grouping:
  Used with grouping attributes (G) to apply functions to subsets of data (e.g., average salary per department).
- Renaming:
  The rename operator (ρ) is used to give meaningful names to the resulting aggregated columns.

COUNT: Counts the number of tuples or non-null values.

SUM: Calculates the total of values in a column.

AVG (Average): Computes the average of values.

MIN: Finds the smallest value in a set.

MAX: Finds the largest value in a set.

Examples:

1. Find total number of students
   $\mathcal{F}$ count(SID)(STUDENT)
2. Find total marks of all students
   $\mathcal{F}$sum(Marks)(STUDENT)
3. $\mathcal{F}$MAX (Salary) (EMPLOYEE) retrieves the maximum salary value from the EMPLOYEE relation
4. $\mathcal{F}$MIN Salary (EMPLOYEE) retrieves the minimum Salary value from the EMPLOYEE relation
5. $\mathcal{F}$SUM Salary (EMPLOYEE) retrieves the sum of the Salary from the EMPLOYEE relation
6. $\mathcal{F}$COUNT SSN, AVERAGE Salary (EMPLOYEE) computes the count (number) of employees and their average salary

Note: count just counts the number of rows, without removing duplicates

Aggregate with grouping
Grouping attribute placed to left of symbol
Aggregate functions to right of symbol
DNO $\mathcal{F}$ COUNT SSN, AVERAGE Salary (EMPLOYEE)

Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department. Result is printed group wise.

Relational Calculus

Tuple Relational Calculus (TRC)

Syntax:$\{t \mid P(t)\}$
t represents a tuple variable.
P(t) is a logical condition (predicate).
The result includes all tuples t that satisfy $P(t)$.
Example
Find names of employees working in the "HR" department.
$\{t.Name \mid t \in Employee \land \exists d \in Department\ (t.Dept\_ID = d.Dept\_ID \land d.Dept\_Name = 'HR')\}$

Domain Relational Calculus (DRC)

Expresses queries using logical conditions on attribute values.
Syntax:
$\{<d1, d2, \ldots, dn> \mid P(d1, d2, \ldots, dn)\}$
Example:
To retrieve the RollNo and Name of students in the CS department, the DRC query is:
$\{<R,N> \mid \exists A,D\ (Student(R,N,A,D) \land D = 'CS')\}$
Find R and N such that there exists A and D where Student(R, N, A, D) is true and D = 'CS'.

Module 3.2

SQL

What is SQL
SQL is a standard Database language which is used to create, maintain and retrieve the data from relational databases.

Advantages

1. High speed
2. No need to have coding skills
3. Standard
4. Portable
5. Interactive

Datatypes

1. Binary:
    a. It has a maximum length of 8000 bytes.
    b. It contains fixed-length binary data.
2. Varbinary:
    a. It has a maximum length of 8000 bytes.
    b. It contains variable-length binary data.
3. Approximate Numeric
    a. Float: can grow as high as 10^{308}
    b. Real: small, stays within 10^{38}
4. Exact
    a. Bit
    b. Int: 4 bytes
    c. Small INT: 2 bytes
    d. Decimal:
    e. Numeric: 5-17 bits of information
       In SQL, DECIMAL = NUMERIC. Both store exact numbers. Both have fixed precision and scale.
5. Date and Time
    a. Used to store date time and month
    b. Date values should be specified in the form: YYYY-MM-DD.
    c. Valid :DATE '1999-01-01’ DATE '2000-2-2’
    d. Invalid: DATE '1999-13-1’ date '2000-2-30’ '2000-2-27’
       date 2000-2-27
       INSERT INTO Orders VALUES(101, ‘`2026-04-30`’)
    e. Time: It is used to store the hour, minute, and second values.
       INSERT INTO Orders VALUES (101, ‘14:30:45');

f. Timestamp: It stores the year, month, day, hour, minute, and the second value.
   INSERT INTO Orders VALUES
   (101, '2026-02-09 14:30:45');
g. Interval: Specifies a range of time
   INSERT INTO Orders VALUES (101, '2026-02-09 14:30:45');

6. Character string datatype
   a. Char: fixed length.
      The length of the char is specified while assigning the data type. For e.g. Char(n). If it’s filled with less than n characters, the remaining are left blank. If it’s filled with more than n characters, it raises an error.
   b. Varchar: variable length
      VARCHAR takes up 1 byte per character, + 2 bytes to hold length information.
   c. Text:
      It has a maximum length of 2,147,483,647 characters.
      It contains variable-length characters.

Logical operators

Operator	Description
ALL	TRUE if all of the subquery values meet the condition
AND	TRUE if all the conditions separated by AND is TRUE
ANY	TRUE if any of the subquery values meet the condition
BETWEEN	TRUE if the operand is within the range of comparisons
EXISTS	TRUE if the subquery returns one or more records
IN	TRUE if the operand is equal to one of a list of expressions
LIKE	TRUE if the operand matches a pattern
NOT	Displays a record if the condition(s) is NOT TRUE
OR	TRUE if any of the conditions separated by OR is TRUE
SOME	TRUE if any of the subquery values meet the condition

Arithmetic (+-/*), Comparison(<>=), Bitwise(&|^), and compound (=/, *, etc) same as C/CPP so skip

Terminologies

Example

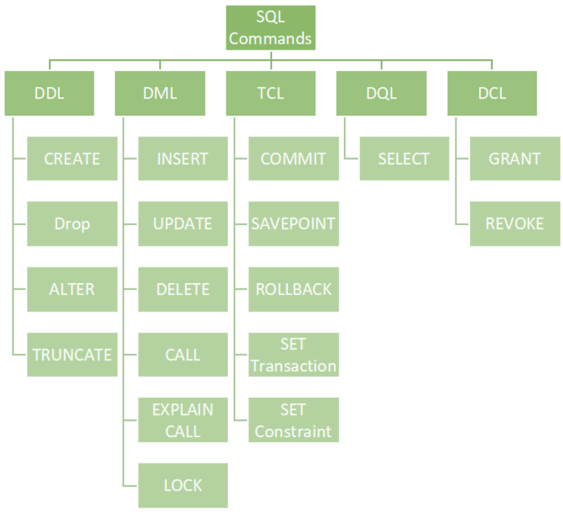
ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	RAM	DELHI	9455123451	18
2	RAMESH	GURGAON	9652431543	18
3	SUJIT	ROHTAK	9156253131	20

Here,

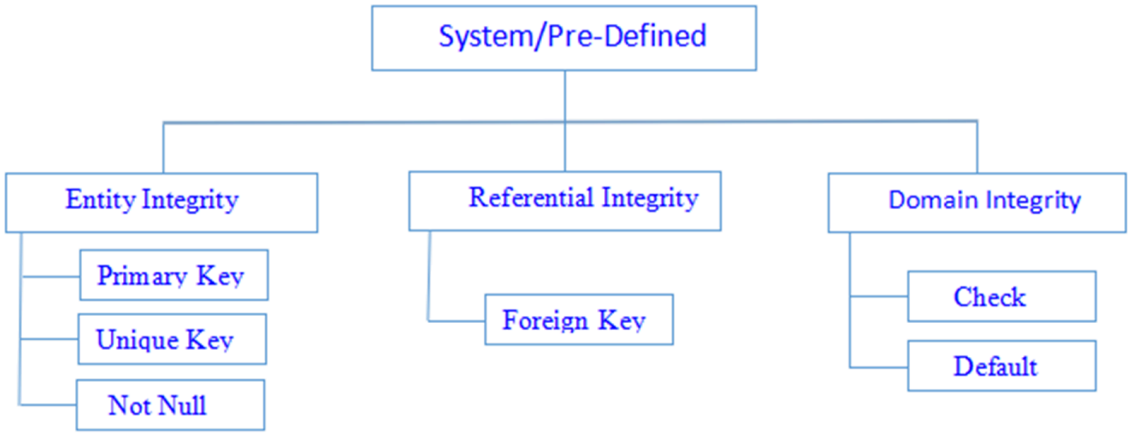
1. Attributes are the properties that define a relation. Example: Name, Roll NO etc
2. Tuple: Each row in the relation is known as tuple.
3. Degree: The number of attributes in the relation. Here, 5
4. Cardinality: number of tuples. Here, 3
5. Column: Column represents the set of values for a particular attribute.
6. A query: It is an inquiry to the database for information.

Classification of SQL Commands

1. DDL (Data Definition Language)
2. DML (Data Manipulation Language)
3. DCL (Data Control Language)
4. DQL (Data Query Language): This command allows getting the data out of the database to perform operations with it.
5. Transactional control commands



Classification of Constraints



Integrity constraints ensure that the changes made to the database are not resulting in a loss of data consistency. Protects the database against accidental damage.

- Constraints can be defined while creating a table
- And also afterwards using ALTER

- Column level definition: Constraints specified immediately after the column definition
- Table level definition: Constraints specified after ALL columns are defined

NOT NULL

- Constraint in a column
- Ensures that no value in that column in NULL
- You will not be allowed to enter a tuple such that the value corresponding to this column is NULL
- Example: Primary Key

Primary Key

- Primary keys must be unique and not null
- Table level declaration:

```
CREATE TABLE Works_On (
    ESSN    INT,
    PNO     INT,
    Hours   FLOAT,
    PRIMARY KEY (ESSN, PNO)
);
```

- In column level, Primary key written with column.
  Example: id INT PRIMARY KEY
- In table level, Primary key written after all columns.
- Both column and table level imply the same thing, only the way of writing is different.
- Moreover, with table level, two columns can be made primary (creation of composite key). This is not possible at the column level.

Foreign key

- FK constraint is used to prevent actions that might destroy a link between two tables
- An FK = PK of the other table
- Table with foreign key = child table
  Table whose PK is FK = parent table
  Example

```
CREATE TABLE customer (
    customerID int NOT NULL,
    FirstName varchar(20),
    LastName varchar(20)
    City int,
```

```
    PRIMARY KEY (customerID),
    FOREIGN KEY (City) REFERENCES City(CityID)
);
Syntax: FOREIGN KEY (column_name) REFERENCES OtherTable(OtherPK)
```

Unique

- Ensures uniqueness
- Both UNIQUE and PRIMARY KEY provide guarantee for uniqueness in a column or set of column
- PK = automatically implies UNIQUE
- You can have multiple UNIQUE constraints in a table but only one PK constraint

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    UNIQUE (ID)
);
```

CHECK

- Used to limit the value range in a column
- Imagine as a drop-down menu. You can only select value per the options presented in the drop-down menu.
- CREATE TABLE abc(
  Age int CHECK(Age>18)
  );

DEFAULT

- Sets a default value for a column
- CREATE TABLE Venue (ID int NOT NULL, Name varchar(255), Age int, Location varchar(255) DEFAULT ‘Mumbai');

Set Operations

- Combine the same type of data from multiple tables
- Different from a join
  - A Join is used to combine columns
  - Set is used to join rows from multiple select queries
- These operators work on complete rows of the queries, so the results of the queries must have the same column name, same column order and the types of columns must be compatible.

UNION

- Combines two or more results in a single result set WITHOUT DUPLICATES
- example
  SELECT <column name> FROM <table name 1>
  UNION
  SELECT <column name> FROM <table name 2>

Table: School1

Standard	Students
First	50
Second	60
Third	40
Fifth	45
Sixth	58
Seventh	77

Table: School2

Standard	Students
First	30
Second	46
Fourth	56
Eight	46
Sixth	34

Output

Standard
Eight
Fifth
First
Fourth
Second
Seventh
Sixth
Third

UNION ALL

- Same as UNION, but allows duplicate rows
- So here, the output would be first, first, second, second, third, fourth, fifth, Sixth, sixth, seventh, eight
- example
  SELECT <column name> FROM <table name 1>
  UNION ALL
  SELECT <column name> FROM <table name 2>

Intersect

- Returns only rows present in both sets
- Select <col> from <table1> INTERSECT SELECT <col> from <table2>
- Output would be first, second, sixth

Except

- Returns all distinct rows present in t1 but not in t2
- Returns the difference between two sets
- SELECT Standard FROM School1
  EXCEPT

SELECT Standard FROM School2
- Result would be first, seventh, third.

NULL

- Field with no value
- Used with optional fields
- Null values cannot be tested with comparison operators likee >, <, =
- To compare, we can only use the IS NULL or IS NOT NULL operator
- Example, email
- SELECT * FROM Employee
  WHERE email IS NULL;

DML: Data Manipulation Language

- Used to manipulate data within objects
- Three basic:
  - Insert
  - Update
  - Delete
- Helps retrieve, insert, delete, and modify data

INSERT

- Used to insert data
- INSERT INTO <TABLE NAME> VALUES
  (<VALUES TO INSERT SEPARATED BY COMMA>)

If you miss a column while inserting values, if it has a default value, it gets that. If it is null, it gets null. otherwise it returns an error.

Update

- Change data of records already in table
- Which rows is to be update, it is decided by a condition. To specify condition, we use WHERE clause.
- Syntax:
  UPDATE students SET Name = ‘Yogesh' WHERE Student_Id = ’5’
- If student id 5 doesnt exist, no change takes place and no error is thrown.

DELETE

- Used to delete rows from a table
- DELETE FROM students where age=“22”;

Aggregation Functions

1. Avg
2. Min
3. Max
4. Total: SUM
5. Count

> Exercise:
>
> Account
>
> | account_number | branch_name | balance |
> |---|---|---|
> | A-101 | Perryridge | 5000 |
> | A-102 | Perryridge | 7000 |
> | A-103 | Downtown | 3000 |
> | A-104 | Perryridge | 8000 |
> | A-105 | Mianus | 4000 |
>
> Depositor
>
> | customer_name | customer_street | customer_city |
> |---|---|---|
> | Adams | Spring St | Perryridge |
> | Brown | Hill St | Mianus |
> | Clark | Park Ave | Downtown |
> | Davis | Lake Rd | Perryridge |
> | Evans | Main St | Mianus |
>
> Customer
>
> | customer_name | account_number |
> |---|---|
> | Adams | A-101 |
> | Brown | A-102 |
> | Clark | A-103 |
> | Adams | A-104 |
> | Davis | A-105 |
>
> - Find the average account balance at the Perryridge branch.
> - Find the number of tuples in the customer relation.
> - Find the number of depositors in the bank.

Complex Retrieval Queries

GROUPING

- Apply aggregate to subgroups of tuples
- Example
  Query 20: For each department, retrieve the department number, the number of employees in the department, and their average salary.
- Q20: SELECT DNO, COUNT (*), AVG (SALARY) FROM EMPLOYEE GROUP BY DNO
  In Q20, the EMPLOYEE tuples are divided into groups-
  Each group having the same value for the grouping attribute DNO
- What this does:
  Say there are three departments a,b,c. The group by aggregate makes sure that the average and count is not printed globally but by taking all accounts of dept a, b, and c separately.
- A join condition can be used in conjunction with grouping

- Examples:
  - Use GROUP BY on single column
  - GROUP BY on multiple columns
  - Use GROUP BY with ORDER BY
  - GROUP BY with HAVING clause
  - Use GROUP BY with JOINS

EmpID	EmpName	EmpEmail	PhoneNumber	Salary	City
1	Nidhi	nidhi@sample.com	9955669999	50000	Mumbai
2	Anay	anay@sample.com	9875679861	55000	Pune
3	Rahul	rahul@sample.com	9876543212	35000	Delhi
4	Sonia	sonia@sample.com	9876543234	35000	Delhi
5	Akash	akash@sample.com	9866865686	25000	Mumbai

Group by on multiple columns

```
SELECT DNO, Gender, COUNT(*) FROM Employee
GROUP BY DNO, Gender;
```

GROUP BY with ORDER BY

The values get sorted by asc/desc order
Example: Write a query to retrieve the number of employees in each city, sorted in descending order.

```
SELECT COUNT(EmpID), City
FROM Employees
GROUP BY City
ORDER BY COUNT(EmpID) DESC;

//displays count of employees per city ordered in descending order
```

HAVING CLAUSE

- Specifies condition on entire group instead of tuples
- We cannot use aggregate functions with WHERE, we use HAVING and GROUP BY

For each project on which more than two employees work, retrieve the project number, project name, and the number of employees who work on that project

```
SELECT PNUMBER, PNAME, COUNT(*)
FROM PROJECT, WORKS_ON
WHERE PROJECT.PNUMBER = WORKS_ON.PNO
GROUP BY PROJECT.PNUMBER, PROJECT.PNAME
HAVING COUNT(*)>2
```

- Write a query to retrieve the number of employees in each city, having salary > 15000

```
SELECT COUNT(EmpID), City
FROM Employees
GROUP BY City
HAVING SALARY > 15000;
```

(Since all are records in the Employee table have a salary > 15000, we will see the following table as output)

Count(EmpID)	City
2	Delhi
2	Mumbai
1	Pune

JOIN

To combine data/ rows from two or more tables.
Four types

Inner

- Returns matching values in both tables
- JOIN is the same as INNER JOIN

```
SELECT table1.column1,table1.column2,table2.column1,...
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

- SELECT Course.COURSE_ID, Student.NAME, Student.AGE FROM Student INNER JOIN Course ON Student.ROLL_NO = Course.ROLL_NO;

This matches the rows in student and course where cid and rno are same and then prints those rows only

Left

- Also called left outer join
- Joins all rows on left with corresponding rows in right. If no corresponding row in right, use NULL for that row
- SELECT Student.NAME,Course.COURSE_ID FROM Student LEFT JOIN Course ON Course.ROLL_NO = Student.ROLL_NO;

Right

- The same as left join
- Joins all rows on right with corresponding rows in left. If no corresponding row in left, use NULL for that row
- SELECT Student.NAME,Course.COURSE_ID FROM Student RIGHT JOIN Course ON Course.ROLL_NO = Student.ROLL_NO;

Full Join

- Combines result of both Left and Right join
- For the rows for which there is no matching, the result-set will contain NULL values.
- SELECT Student.NAME, Course.COURSE_ID FROM Student FULL JOIN Course ON Course.ROLL_NO = Student.ROLL_NO;

DCL

- DCL includes commands such as GRANT and REVOKE which mainly deal with the rights, permissions, and other controls of the database system.
- GRANT name_of_privilege ON object TO user
    - name_of_privilege: These are the access rights or privileges granted to the user.
    - object: It is the name of the database object to which permissions are being granted. In the case of granting privileges on a table, this would be the table name.
    - user: It is the name of the user to whom the privileges would be granted.

Example
GRANT SELECT ON Users TO 'Amit'@'localhost’;

This statement grants the user called Amit the privilege to select on the table called “Users”
To give multiple privileges, separate with comma
GRANT SELECT,DELETE,INSERT ON Users TO 'Amit'@'localhost’;

Extra:
@host specifies where user connects from.

'Amit'@'localhost' → only from same machine.
'Amit'@'%' → from any host.

- REVOKE: withrdraws user permissions
- Opposite of grant

- REVOKE privileges ON object FROM user
- revoke insert, select on accounts from Ram

Alias

- Temporary name given to tables/columns for a particular query
- The renaming does not affect the column name in the table in database
- SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME
  FROM EMPLOYEE AS E, EMPLOYEE AS S
  WHERE E.SUPERSSN=S.SSN
- We can think of E and S as two different copies of the same table EMPLOYEE; E represents employees in role of supervisees(employee) and S represents employees in role of supervisors
- Column ALIAS: SELECT column as alias_name FROM table_name;
- Example: SELECT CustomerID AS SSN FROM Customer;
- Table ALIAS: SELECT s.CustomerName, d.Salary FROM Customer AS s, Department AS d WHERE s.Age=21 AND s.DNO=d.DNO;

DISTINCT

- Used to eliminate duplicate tuples in a query result
- Example usage
- SELECT DISTINCT SALARY FROM EMPLOYEE

Logical Operators

LIKE

- Performs pattern matching to find correct result
- Used in SELECT, INSERT, UPDATE, DELETE with combination of WHERE
- Syntax: expression LIKE pattern [ESCAPE 'escape_character']
  - expression: It specifies a column or field.
  - pattern: It is a character expression that contains pattern matching.
  - escape_character: It is optional. It allows you to test for literal instances of a wildcard character such as % or _. If you do not provide the escape_character, MySQL assumes that "\" is the escape_character.

Example

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

AND, OR, NOT

1. AND, OR: works like boolean AND and OR.
   If both conditions are true, AND displays a record
   OR displays a record if any one condition is true
2. NOT
   Displays a record if condition is not true

Example

```
mysql> select * from stud where not address='Delhi';
+------+--------+----------+------+
| rno  | name   | address  | age  |
+------+--------+----------+------+
|    2 | Pratik | Bihar    |   19 |
|    4 | Deepak | Ramnagar |   18 |
|    5 | Niraj  | Kolkata  |   19 |
+------+--------+----------+------+
3 rows in set (0.00 sec)
```

Example

```
mysql> select * from stud where address='Delhi' AND age=18;
+------+-------+---------+------+
| rno  | name  | address | age  |
+------+-------+---------+------+
|    1 | Harsh | Delhi   |   18 |
+------+-------+---------+------+
1 row in set (0.00 sec)
```

IN

- Allows to specify multiple values in WHERE

```
mysql> select * from stud where rno in (select rno from course1);
+------+--------+----------+------+
| rno  | name   | address  | age  |
+------+--------+----------+------+
|    1 | Harsh  | Delhi    |   18 |
|    2 | Pratik | Bihar    |   19 |
|    4 | Deepak | Ramnagar |   18 |
+------+--------+----------+------+
3 rows in set (0.00 sec)
```

```
mysql> select * from stud where address IN('Delhi','Kolkata');
+------+--------+---------+------+
| rno  | name   | address | age  |
+------+--------+---------+------+
|    1 | Harsh  | Delhi   |   18 |
|    7 | Mahesh | Delhi   |   20 |
|    5 | Niraj  | Kolkata |   19 |
+------+--------+---------+------+
3 rows in set (0.00 sec)
```

Between

- Selects value in a range
- Can be numbers texts or dates
- Inclusive: begin & end values are included

```
mysql> select * from stud where age between 19 and 20;
+------+--------+---------+------+
| rno  | name   | address | age  |
+------+--------+---------+------+
|    2 | Pratik | Bihar   |   19 |
|    7 | Mahesh | Delhi   |   20 |
|    5 | Niraj  | Kolkata |   19 |
+------+--------+---------+------+
3 rows in set (0.00 sec)
```

Any

- The ANY operator is used to verify if any single record of a query satisfies the required condition
- SELECT * FROM CUSTOMERS WHERE SALARY > ANY (SELECT SALARY FROM CUSTOMERS WHERE AGE = 32);
- This selects the records in the customer table where the salary is more than any salary in the database where the customer age is greater than 32.

ALL

- Returns true if the condition is satisfied for all in that range
- SELECT * FROM CUSTOMERS WHERE SALARY <> ALL (SELECT SALARY FROM CUSTOMERS WHERE AGE = 25);
- This selects the records in the customer table where the salary is not equal to(<>) any salary in the database where the customer age is 25.

VIEW

- It is a virtual table
- Represents a subset of a real table, only having some columns/rows
- Because views are assigned separate permissions, you can use them to restrict table access so that the users see only specific rows or columns of a table.
- syntax
  CREATE [TEMP|TEMPORARY] VIEW view_name
  AS SELECT column1, column2.....
  FROM table_name WHERE [condition];
  when temp/temporary is mentioned, the view is created temporarily and is dropped at the end of the current session.
- Example
  CREATE VIEW COMPANY_VIEW
  AS SELECT ID, NAME, AGE
  FROM COMPANY;
  This would create a view called company_view that would have id, name, and age of all people in company
- To drop a view after completion, we write
  DROP VIEW COMPANY_VIEW

Advantages

1. Simplifies hard queries
   a. You can create simple views based on complex queries first and then directly work on them
2. Security
   a. Can create views that show subsets of data and hide away sensitive info
3. Logical data independence
   a. Views provide logical data independence by hiding table changes from users.

Updating a view

- A view can be updated under certain conditions which are given below –
- The SELECT clause may not contain the keyword DISTINCT.
- The SELECT clause may not contain summary functions.
- The SELECT clause may not contain set functions.
- The SELECT clause may not contain set operators.
- The SELECT clause may not contain an ORDER BY clause.
- The FROM clause may not contain multiple tables.
- The WHERE clause may not contain subqueries.
- The query may not contain GROUP BY or HAVING.
- Calculated columns may not be updated.
- All NOT NULL columns from the base table must be included in the view in order for the INSERT query to function.

Inserting

1. Rows can be inserted in view
2. Update rules apply to insert also
3. Cannot insert a row if all not null columns are not included in the view.

Deleting

1. Rows can be deleted
2. Same rules as insert apply
3. DELETE FROM CUSTOMERS_VIEW WHERE age = 22;
4. This deletes the row from base table and hence reflects change in view

Creating view from multiple tables

1. View from multiple tables can be created by adding more tables after select
2. CREATE VIEW MarksView AS SELECT Student_Detail.NAME, Student_Detail.ADDRESS, Student_Marks.MARKS FROM Student_Detail, Student_Mark WHERE Student_Detail.NAME = Student_Marks.NAME;

Trigger

Triggers are automatic actions that execute when an INSERT, UPDATE, or DELETE happens on a table.
We have them to enforce rules, maintain logs, or update other tables automatically

1. Automatically runs when an event occurs in the database server
2. DML triggers when a user tries modifying data
3. When a modification is made, these triggers fire when any valid event fires, whether table rows are affected or not.
4. BEFORE: Triggers run the trigger action before the triggering statement is run.
5. AFTER: triggers run the trigger action after the triggering statement is run.

Syntax of trigger func:

```
CREATE FUNCTION trigger_func()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
{LOGIC}
END;
$$
```

Example

```
CREATE OR REPLACE FUNCTION data_log()
  RETURNS TRIGGER
  LANGUAGE PLPGSQL
  AS
$$
BEGIN
INSERT INTO main(Backup_id,Backup_name)
         VALUES(OLD.ID,OLD.name);

RETURN NEW;
END;
$$
```

```
create trigger data_backup
after delete
on origin
for each row
execute procedure data_log();
```

Nested Queries

- A complete query can be specified within the WHILE clause of the outer query.
- Subqueries can be used with the SELECT, INSERT, UPDATE, and DELETE statements along with the operators like =, <, >, >=, <=, IN, BETWEEN, etc.

Query 1: Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:SELECT     FNAME, LNAME, ADDRESS
   FROM       EMPLOYEE
   WHERE      DNO IN
      (SELECT  DNUMBER
          FROM        DEPARTMENT
          WHERE       DNAME='Research' )
```

Correlated Nested Queries

- If a condition in the WHERE-clause of a nested query references an attribute of a relation declared in the outer query, the two queries are said to be correlated
- Correlated nested queries are subqueries that depend on the outer query.
- The **CONTAINS** operator compares two sets of values, and returns TRUE if one set contains all values in the other set

- Query 3: Retrieve the name of each employee who works on all the projects controlled by department number 5.

```
Q3:     SELECT      FNAME, LNAME
        FROM        EMPLOYEE
        WHERE  (    (SELECT     PNO
                     FROM       WORKS_ON
                     WHERE      SSN=ESSN)
                           CONTAINS
                    (SELECT     PNUMBER
                     FROM       PROJECT
                     WHERE      DNUM=5) )
```

- **EXISTS** checks whether the result of a correlated nested query is empty.

Query 12: Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
Q12B:   SELECT      FNAME, LNAME
        FROM        EMPLOYEE
        WHERE       EXISTS  (SELECT    *
                             FROM       DEPENDENT
                             WHERE      SSN=ESSN
                        AND
    FNAME=DEPENDENT_NAME)
```

TCL: Transaction Control Languages

- Maintains consistency
- Manages transactions made by DML
- Transaction: Set of SQL statements that are executed on the data
- These transactions are temporary. To make them permanent, we need TCL commands.
- **Commit:** This command is used to save the data permanently. DML commands are rolled back if not committed.

- **Rollback:** Used to get the data/restore the data to the last savepoint. Only temporary data is rolled back. A committed change cannot be rolled back.
- **Savepoint:** saves the data at a particular point temporarily so that the rollback to that particular point.
  Syntax:
  SAVEPOINT A

Example

```
BEGIN;

    INSERT INTO Employee VALUES (101, 'Alice', 50000);
    SAVEPOINT sp1;

    UPDATE Employee SET Salary = 60000 WHERE EID = 101;
    SAVEPOINT sp2;

    DELETE FROM Employee WHERE EID = 101;

ROLLBACK TO sp2;  -- undoes DELETE, goes back to after UPDATE

ROLLBACK TO sp1;  -- undoes UPDATE, goes back to after INSERT

COMMIT;  -- saves everything up to this point
```

Cursor

In DBMS, a cursor is a database object used to retrieve or manipulate data row by row, especially when dealing with result sets that consist of multiple rows.

Cursors are particularly useful when you need to work with data sequentially, process one row at a time, or when you need to perform operations that can't be easily accomplished with set-based operations.

a. Implicit:
   Implicit cursors are managed by the DBMS internally, and you don't need to explicitly declare or control them in your code. Automatically created for CRUD operations
b. Explicit:
   Explicit cursors provide more control over the result set traversal, allowing the programmer to open, fetch, and close the cursor as needed.

Declare

```
DECLARE cursor_name CURSOR FOR
      SELECT col1, col2
      FROM table_name,
      WHERE condition;
```

Example

```
DECLARE
    my_cursor CURSOR FOR
    SELECT NAME, AGE FROM Student;
```

Opening

```
OPEN cursor_name;
```

Fetching

```
FETCH cursor_name into variable1, variable2
```

Example

```
FETCH my_cursor INTO v_name;
```

Processing

After fetching a row, you can perform operations or calculations using the retrieved data. This step allows you to apply business logic or perform calculations on a row-by-row basis.

Closing

```
CLOSE cursor_name;
```

- Consider the following employee database.
- Employee(emp_name, street,city,date_of_joining)
- Works(emp_name,company_name,salary)
- Company(company_name,city)
- Manages(emp_name,manager_name)
- Write SQL queries for following:

1. Modify the database so that 'Deepa'lives in 'Pune';
2. Give all employees of 'Aarya corporation' a 10% rise in salary.
3. Display all employees who joined in the month of 'March';
4. Find all employees who earn more than average salary of all employees of their company.

Functions and Stored Procedures

Function/procedure definition:
A group of SQL statements that perform a task. A function must always return a value but a procedure may or may not return a value.

Sr. No.	Key	Function	Procedure
1	Definition	A function is used to calculate result using given inputs.	A procedure is used to perform certain task in order.
2	Call	A function can be called by a procedure.	A procedure cannot be called by a function.
3	DML	DML statments cannot be executed within a function.	DML statements can be executed within a procedure.
4	SQL, Query	A function can be called within a query.	A procedure cannot be called within a query.
5	SQL, Call	Whenever a function is called, it is first compiled before being called.	A procedure is compiled once and can be called multiple times without being compiled.
6	SQL, Return	A function returns a value and control to calling function or code.	A procedure returns the control but not any value to calling function or code.

Function

Stored program **that returns a value.**

Can return
   1. A table
   2. A value
   3. A record
   4. A set of rows
Usually used inside the SELECT statements.

```
CREATE OR REPLACE FUNCTION function_name (arguments)
RETURNS return_datatype
language plpgsql
AS $$
declare
variable_name -- variable declaration
begin
-- stored function body
end; $$
```

Example
A Function to Calculate Square of a Number:

```
CREATE FUNCTION square(num INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN num * num;
END;
$$;
```

Call the function:

```
SELECT square(5); -- Output: 25
```

Procedures

- A procedure is a named PL/SQL block which performs one or more specific tasks.
- CREATE OR REPLACE PROCEDURE procedure_name (parameters)
  AS $$
  BEGIN
  -- Procedure logic
  END;
  $$ LANGUAGE plpgsql;
- To call, use CALL procedure_name(parameter(s));
- **Does not have to return a value**

Example

```
CREATE PROCEDURE raise_salary(EID INT, amount INT)
AS $$
BEGIN
    UPDATE Employee
    SET Salary = Salary + amount
    WHERE Employee.EID = EID;
END;
$$ LANGUAGE plpgsql;
```

Module 4.1

Normalization

Definition:

- A technique of organizing data into multiple related tables.
- Helps reduce data redundancy.
- It is the process of decomposing bad relations by breaking their attributes into smaller relations.

Example

StudentID	StudentName	Course	Faculty
1	Ali	DBMS	Khan
1	Ali	OS	Ahmed
2	Sara	DBMS	Khan

This is a bad table since Ali and Khan are repeated. Data redundancy is observed.
If instead we reduce this into normalized tables, we get:

StudentID	StudentName
1	Ali
2	Sara

And

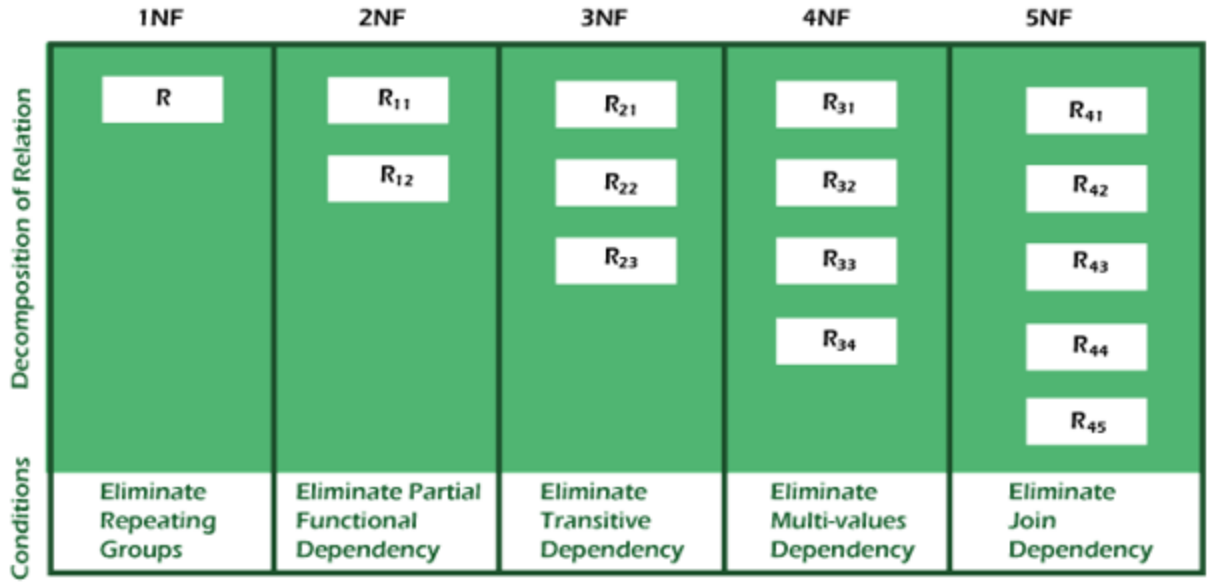
Course	Faculty
DBMS	Khan
OS	Ahmed

Three benefits: Less redundancy. Easy updates. Better consistency.

Normal form

Normalization: The process of decomposing unsatisfactory relations by breaking up their attributes.

Normal form: A normal form is a rule or condition used to determine the level of normalization of a relation schema. It is a standard that tells whether a table is properly organized to reduce redundancy and dependency problems.



1NF

Each cell stores one single value, no repeating groups.
1nf Forbids

1. Composite attributes
2. Multivalued attributes
3. Nested relations: attribute in the same row are a tuple

Bad (not 1NF):
Student | Subjects
A | Math, Science

Here one cell has many values.

Good (1NF):
Student | Subject
A | Math
A | Science

1NF forbids composite attributes, multivalued attributes, and nested relations.

2NF

It eliminates partial functional dependency.
For example

StudentID	CourseID	StudentName	CourseName
1	C1	Ali	DBMS
1	C2	Ali	OS

Here, student name only depends on studentID and coursename only depends on courseID. These are partial dependencies. In 2nf, every non-key attribute must depend on the whole primary key, not just part of it. Here primary key = (StudentID, CourseID), but all other columns do not depend on BOTH of these primary keys. Therefore, the table is not in 2NF.

Correct 2nf form for the above table:
Student table
Primary key: StudentID
Dependency: StudentID → StudentName
StudentID	StudentName
1	Ali

Course table
Primary key: CourseID
Dependency: CourseID → CourseName
CourseID	CourseName
C1	DBMS
C2	OS

Enrollment table
Enrollment Table
Primary key: (StudentID, CourseID)
Since there are no non-key attributes here, so no partial dependency.
StudentID	CourseID
1	C1
1	C2

****NOTE:** key attributes = attributes that constitute the primary key.
Non key attributes = attributes that depend on the primary key and are not a part of the primary key.

3NF

It eliminates transitive dependency.

For example

StudentID	DeptID	DeptName
1	D1	CS
2	D2	IT

Here, the primary key is studentID
Dependencies: StudentID → DeptID, DeptID → DeptName
This follows a transitive relation from a->b and b->a.
Therefore, this table is not in 3NF
To bring a table in 3NF, we need to get rid of all transitive dependencies.
The solution:
Student table

StudentID	DeptID
1	D1
2	D2

Department table

DeptID	DeptName
D1	CS
D2	IT

Now: StudentID → DeptID
DeptID → DeptName

No transitive dependency inside the same table. Tables are in 3nf. In both tables, every non-key attribute depends directly on the primary key only.

BCNF

A table is in BCNF if:
for every functional dependency, X → Y, X must be a super key.

What is a superkey?
A super key is any attribute or set of attributes that can uniquely identify a row in a table.

For example

Student	Subject	Teacher
Ali	DBMS	Khan
Sara	DBMS	Khan

Here, Dependencies:
(Student, Subject) → Teacher
Teacher → Subject
Subject → Teacher
Because by knowing the value of the LHS alone you can determine the value of RHS.

What are candidate keys?
Candidate keys are the smallest sets of attributes that can uniquely identify each row.
Here, Candidate keys:
{Student, Subject}
{Student, Teacher}

Since Teacher determines Subject but Teacher is not a super key, the table is not in BCNF.
Solution:
Decompose the table into

Teacher	Subject
Khan	DBMS

And

Student	Teacher
Ali	Khan
Sara	Khan

Now, all non-key attributes depend on the superkey.

4NF

4NF eliminates the multi-valued dependencies. A table is in 4nf iff it is in BCNF and has no multivalued dependencies.

For example

Student	Hobby	Language
Ali	Cricket	English
Ali	Cricket	Hindi
Ali	Music	English
Ali	Music	Hindi

Here, student can have independently many hobbies and can know independently many languages. Since they are independent, database creates all combinations. This unnecessary mixing is called multivalued dependency.
Decompose into

Student	Hobby
Ali	Cricket
Ali	Music

And

Student	Language
Ali	English
Ali	Hindi

5nf eliminates any join dependency. Detail for the same is not in syllabus.

Normal Form	One line explanation	Without normal form	After applying normal form
1NF	No multivalued or repeating attributes	Student(ID, Name, Phones{999,888})	Student(ID, Name, Phone)
2NF	No partial dependency on composite key	(StudentID, CourseID, StudentName, CourseName)	Student(StudentID, StudentName), Course(CourseID, CourseName), Enrollment(StudentID, CourseID)
3NF	No transitive dependency	(StudentID, DeptID, DeptName)	Student(StudentID, DeptID), Department(DeptID, DeptName)
BCNF	Every determinant must be a super key	(Student, Subject, Teacher) with Teacher → Subject	Teacher(Teacher, Subject), StudentTeacher(Student, Teacher)
4NF	No independent multivalued dependencies	(Student, Hobby, Language)	StudentHobby(Student, Hobby), StudentLanguage(Student, Language)

Denormalization

The process of storing the join of higher normal form relations (4nf, 5nf)as a base relation—which is in a lower normal form (3nf, 2nf).
Simpler: Denormalization is the process of taking multiple normalized tables and combining them back into one table.

Key (superkey):

A set of columns that can uniquely identify each row

A key K is a superkey if the removal of any attribute from K will cause K not to be a superkey any more. Example: if a table depends on both student name and course name and we remove the student name from the picture, the courseName alone cannot uniquely identify the row. Therefore, the entire superkey ceases to exist.

Candidate Key

1. If a relation schema has multiple keys, each key is called a candidate key.
2. One candidate key is arbitrarily designed to be the primary key
   Others, secondary keys

Prime attribute

Must be a member of some candidate key

Non prime attribute

Not a member of any candidate key.

Anomalies

- Insertion anomaly: You cannot add new data without adding unwanted/redundant data.
- Deletion anomaly: Loss of related dataset when some other dataset is deleted.
- Updation anomaly: Say a table has the same column value for multiple rows. For example, the HOD changed from MrP to Mr D. Now while updating, you might miss a row, causing inconsistency

Decomposition

The process of decomposition in DBMS helps reduce redundancy, inconsistency and anomalies.

Lossless Join Decomposition

Lossless join decomposition is when the join of the sub relations results in the same relation R that was decomposed to give the subrelations.

Lossy Join Decomposition

A lossy join decomposition is when the join of the sub relations does not result in the same relation R that was decomposed.

To determine whether a relation is lossless or lossy, check for the following conditions. If any one fails, the decomposition is lossy.

1. The union of both sub relations must contain all attributes in the original relation.
2. Intersection must not be null
3. Intersection must be a superkey of either r1, or r2, or both

All the decomposition we perform must be lossless. All information must be retained while performing the original relation.

Dependency preservation: Ensures that the functional dependencies between entities is maintained while we perform decomposition. It helps make the database more efficient and maintains its integrity and consistency.
Data Redundancy Lack: Decomposition should not suffer from redundant data. Helps get rid of unwanted data and only retain useful data.

Example question:
Consider a relation schema R (A , B , C , D) with the following functional dependencies_x0002_
A → B
B → C
C → D

Determine whether the decomposition of R into R1 (A , B) , R2 (A , C) and R3 (B , D) is
lossless or lossy.

Relational Database Design

What is a relational database design?
The grouping of attributes to form a good relation schema.
There are two levels of relation schemas:

1. Logical user-view level
2. Storage base-relation level.

Guidelines:

- **Attributes of different entities should not be mixed up** in the same keys to refer to the other entities. Entity and relationship attributes should be kept as separately as possible.
- The designed schema should not suffer from any **anomalies**.
- Relations should be designed so that tuples have **very few NULL values.** Attributes that are usually NULL should be kept separately in another table because if a NULL value comes in comparison with SELECT or JOIN, the result becomes unpredictable.
- **Spurious Tuples:** Spurious tuples are unwanted extra rows formed after joining decomposed tables incorrectly.
  Example:

A	B	C
1	X	P
2	X	Q

  If we decompose this incorrectly into

A	B
1	X
2	X

  And

B	C
X	P
X	Q

  This could imply that 1->x->q is a relation which is false.
  Therefore, relations should be designed such that they satisfy the lossless join condition and no spurious tuples are generated
- While decomposing, **losslessness and non-additiveness must be preserved**. Functional dependencies must be preserved but can be sacrificed sometimes.

Functional Dependencies

- Defines how good a relational design is
- FDs are rules about data. They show which column depends on which other column.
- Keys are used to define normal forms of relations
- Constraints are derived from the meanings and interrelationships of data attributes.

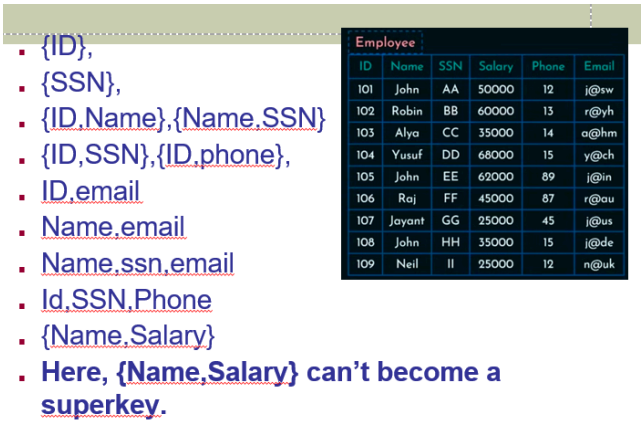
Example:
Constraints:
RollNo must be unique.
Dept cannot be null.

Functional Dependency:
RollNo → Name, Dept
Means that if you know RollNo, you can uniquely determine Name and Dept.

Key:
RollNo is the primary key.

Keys

1. Superkey
    a. All possible combination of keys = superkeys
    b. Used to identify record
    c. It is the superset. We can use it to derive other keys.



    d. This is because name and salary are not unique across all tuples. Hence, they cannot be used to uniquely identify a record
2. Candidate key
    a. Smallest set of superkeys that can uniquely identify a row.
    b. Candidate keys other than primary keys are called alternate keys.
3. Composite Key

   a. Combination of multiple attributes
   b. Example: {Name,Phone} {ID,Phone} {ID,Name} etc.
4. Unique key
   a. Should contain unique values
   b. But can be null
   c. Example: phone number or email
5. Foreign key
   a. Field that establishes link between two tables
   b. Acts as a constraint ensuring that data and references maintain integrity

Proper subset: A is a proper subset of B if it is a subset of B and does not have all the elements in B.

Functional Dependencies

- X -> Y is a functional dependency that says Y depends on X. It holds if whenever two tuples have the same value for X, they must have the same value for Y.
- Example: Social security number determines employee name. SSN -> ENAME
- X is called Determinant, while on the right side, Y is called the Dependent.
- The functional dependency is a relationship that exists between two attributes, where one set can accurately determine the value of other sets.

Inference Rules

1. (Reflexive) If Y subset-of X, then X -> Y
2. (Augmentation) If X -> Y, then XZ -> YZ
   XZ stands for X union Z
3. (Transitive) If X -> Y and Y -> Z, then X -> Z

All other inference rules can be deduced from the above three.

4. Decomposition: If X -> YZ, then X -> Y and X -> Z
5. Union: If X -> Y and X -> Z, then X -> YZ
6. Pseudotransitivity: If X -> Y and WY -> Z, then WX -> Z

Closure = “everything you can get”. All things that can be derived or determined from a given set.

Two types:

1. FD closure (F^+):
   All FDs you can derive from given F.
2. Attribute closure (X^+):
   All attributes you can reach from X using FDs from F.

Types of Functional Dependencies

1. Full dependency
   Y depends on X completely.
   Remove any part of X → dependency breaks.
   Key = (StudentID, CourseID)
   (StudentID, CourseID) → Grade
   Grade needs both attributes → full dependency.
2. Partial dependency
   Non-key attribute depends on only one part of a composite key.
   Example:
   Key = (StudentID, CourseID)
   StudentID → Name
   Name depends only on StudentID → partial.
3. Trivial dependency
   Dependent is a subset of the determinant.
   If x->y, then y is a subset of x
   {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name
4. Non-trivial dependency
   The dependent is strictly not a subset of the determinant
   x->y, y cannot be a subset of x.
   roll_no → name is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no
5. Multivalued dependency
   Attributes of the dependent sets are not dependent on each other.
   If a → {b, c} and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.
6. Transitive dependency
   If a->b and b->c then a->c holds, then it is a transitive functional dependency.

Equivalence of Sets of FDs

Two sets of FDs F and G are equivalent if

1. Every FD in F can be inferred from G
2. Every FD in G can be inferred from F

This implies that you can get to G from F and vice versa by the functional dependencies.

3. F and G are equivalent if $F^+ = G^+$

Consider a relation:
R(A, B, C)
Set F, F = {A → B,B → C} Set G, G = {A → B,A → C}

Check whether F and G are equivalent
Step 1: Find F^+ (closure of F)
Given: A → B, B → C
Using transitivity rule: A → B, B → C Therefore: A → C
F^+ = { A→B, B→C, A→C }
Check if G can derive F
G = { A → B, A → C }
Check if B → C can be derived.
From G: A → B, A → C
But B → C cannot be derived.
So $G^+ \neq F^+$.
Therefore:
F and G are NOT equivalent.

Superkeys

1. If we have N attributes, and 1 candidate key, then we have 2^{n-1} possible superkeys.
   Say we have a relation with attributes a1, a2, and a3. Then any superset of a1 is the superkey if a is a primary key.
   four possible super keys: {a1, a1 a2, a1 a3, a1 a2 a3}.
2. Find closure of E-ID:
   Start: S = {E-ID}
   Now add what it gives:
   Use FDs: E-ID → E-NAME → add E-ID → E-CITY → add E-ID → E-STATE → add
   Now S = {E-ID, E-NAME, E-CITY, E-STATE}
   Check more: E-CITY → E-STATE (already present) Final: (E-ID)+ = {E-ID, E-NAME, E-CITY, E-STATE}
   E-CITY also gives E-STATE
   but you already have it.
   Final answer:
   $E\text{-}ID^+$ = all attributes
   Meaning:
   E-ID alone finds the whole row.

Minimal Cover

A minimal cover of a set of FDs is the smallest equivalent set of FDs with no redundancy.
3 Conditions for Minimal Cover

- RHS of every FD has single attribute
- No redundant FD exists
- No redundant attribute on LHS

Steps to Find Minimal Cover

Step 1 – Simplify RHS (one attribute each)
A → BC becomes A → B and A → C
Step 2 – Remove redundant attributes from LHS
AB → C check if A → C alone works
if yes, remove B
Step 3 – Remove redundant FDs
Check if each FD can be derived from remaining FDs
If yes, remove it

Quick Example
F = {A → BC, B → C, A → B}

Step 1: A → B, A → C, B → C, A → B

Step 2: No composite LHS, skip

Step 3: A → C is redundant
        (A → B and B → C gives A → C)
        Remove A → C

Minimal Cover = {A → B, B → C}

Module 4.2

Indexing

What is an SQL index?

- Quick lookup table to find records quickly
- Very useful for connecting relational tables and searching in large tables
- Index speeds up the SELECT queries and the WHERE clauses. It slows down the data input like UPDATE and INSERT because now there's one more cell to change

SQL supports multiple index types but the most common is clustered. This type of index is automatically created with primary keys.
Indexing is a way to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.

Attributes of indexing

1. Access Types: This refers to the type of access such as value based search, range access, etc.
2. Access Time: It refers to the time needed to find a particular data element or set of elements.
3. Insertion Time: It refers to the time taken to find the appropriate space and insert new data.
4. Deletion Time: Time taken to find an item and delete it as well as update the index structure.
5. Space Overhead: It refers to the additional space required by the index.

Dense vs Sparse Indexes

Dense index: index contains an entry for every record/search key. Searching is faster but it leads to more space overhead.
Example:
Table:
1 Ali
2 Sara
3 John
Dense index:
1 → row1
2 → row2
3 → row3

Sparse index: index contains entries for only some records. Needs less space and less maintenance for insertion and deletion but it’s slower than dense index.
Example:
1 → first block
3 → another block
Database searches nearest indexed entry, then scans nearby records.

Types of Indices

The index typically stores each value of the index field along with a list of pointers to all disk blocks that contain records with that field value.

Primary Index

Primary index is defined on an ordered data file, based on the PK of the relation. A primary index is an ordered file. It has a record of size two. The first field of the record is of the same datatype as the ordering key field (the index itself) and the other field is a pointer to a disk block.

Cluster Index

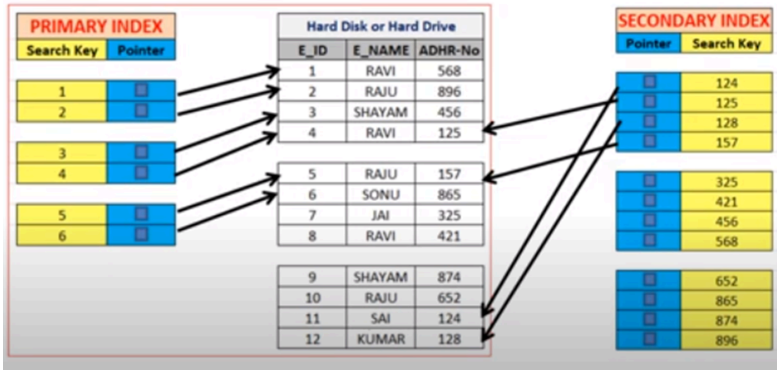
If the ordering field is not a key field—that is, if numerous records in the file can have the same value for the ordering field, we use cluster index.
A cluster index is defined on an ordered data file, based on a non key field.
A file can have at most one primary index or one clustering index, but not both.
Records with similar characteristics are grouped and an index is created for this group. The ordered file in this kind of index has two fields: one is the ordering key field and the other is a disk block pointer.
e.g. all employees of same dept grouped together

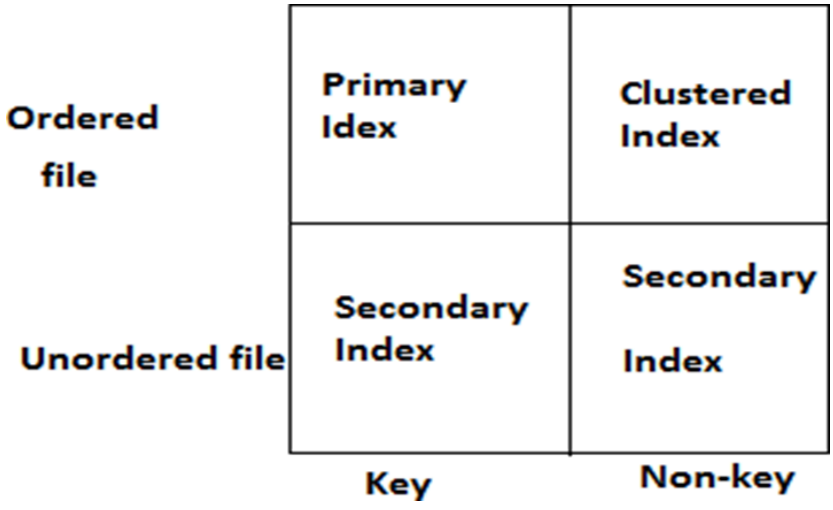
Clustered Index	Non-Clustered Index
Physically stores table data in index order	Separate structure from actual table data
Changes actual row order	Does not change row order
Only one possible per table	Multiple possible per table
Faster for range searches	Faster for exact lookups
Leaf nodes contain actual data rows	Leaf nodes contain pointers to rows
Example: sorting students table by RollNo	Example: index on Name column

Secondary Index

Defined in an unordered data file, based on a candidate key. A data file can have multiple secondary indexes along with either the primary/clustered index.
Also called non clustered indexing.
We use this when we have unordered data, want to search based on key or non key attributes, and the data already has a primary index.

- can be generated by a unique key which is also a candidate key
- If mapping size increases, fetching address becomes slow. Therefore, sparse indexing becomes inefficient. To overcome this we have secondary index.
- To reduce mapping size, we have another level of indexing. In this method, a huge range of columns is initially selected so that the mapping size originally is small. Then each range is divided into smaller ranges.
- The mapping of the first level = primary memory. So address fetch is quick
- Mapping of second level = actual data stored in secondary memory





Unique Index

A UNIQUE constraint is mainly for data integrity. It guarantees no duplicate values in a column.
A UNIQUE index also prevents duplicates, but is more about faster searching.
Internally, both usually work similarly, so query performance is almost same.

When you make a PRIMARY KEY, DBMS usually creates a unique clustered index automatically.
Meaning:
table data itself gets physically ordered by that key.

When you make a UNIQUE constraint, DBMS usually creates a unique non-clustered index.
Meaning:
separate index structure is created, table order unchanged.

You cannot create a UNIQUE constraint/index if duplicates already exist.

MultiLevel Index

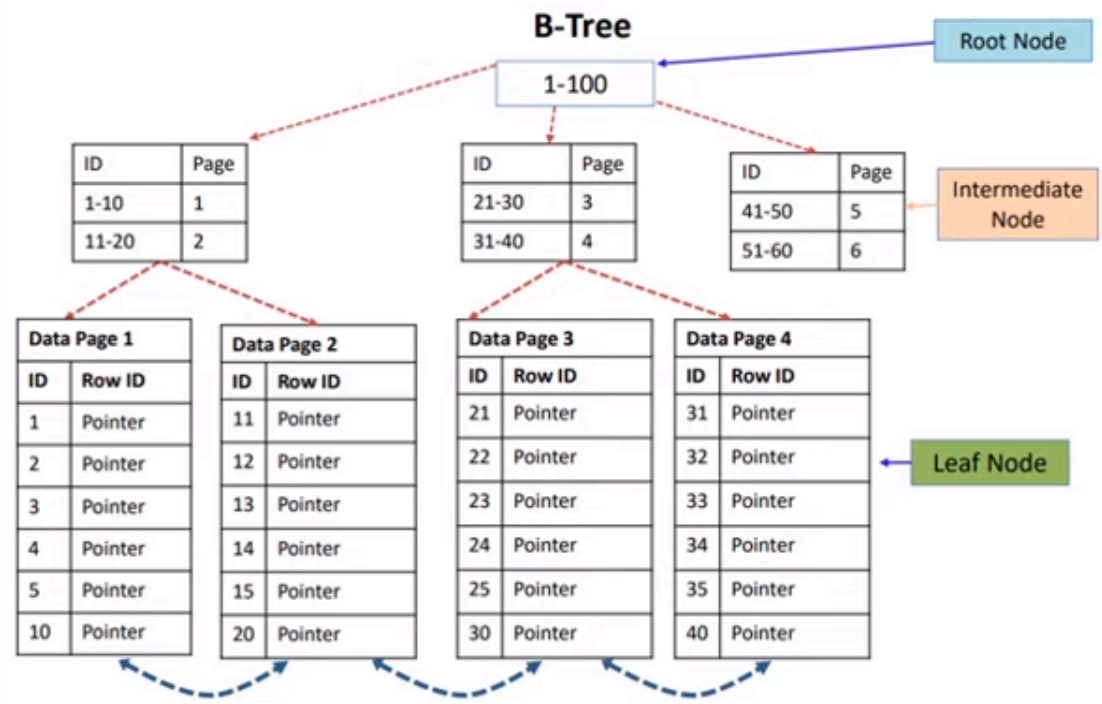
Multilevel indices are indexes built on top of other indexes to make searching faster in large databases.

Index records comprise search-key values and data pointers. Multilevel index is stored on the disk along with the actual database files. As the size of the database grows, so does the size of the indices. There is an immense need to keep the index records in the main memory so as to speed up the search operations. If a single-level index is used, then a large size index cannot be kept in memory which leads to multiple disk accesses. Multi-level Index helps in breaking down the index into several smaller indices in order to make the outermost level so small that it can be saved in a single disk block, which can easily be accommodated anywhere in the main memory.

Say you have roll numbers in batches divided between four divisions A to D
You want to look for roll 107
You know that range for B div is 76 to 140
So you will not check a,c,d and directly go to b
Then search for 107 in the 70 roll numbers in b
Two levels: division and roll

B-tree

B trees are self balancing trees as the nodes are sorted in the inorder traversal.
A btree can have nodes with more than two children. It is not a binary tree.
Height of a b tree is Log_m n where M = order of tree(max number of children in the tree) and N = number of nodes

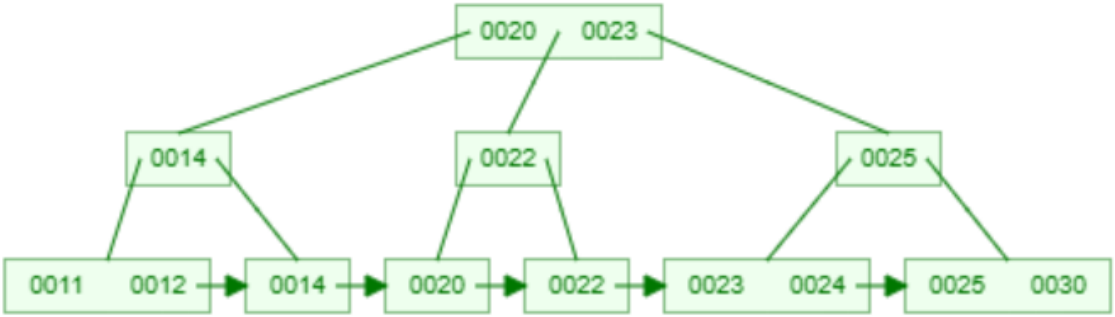


B+ Tree

A B+ tree is a balanced binary search tree that follows a multi-level index format. The leaf nodes denote the data pointers. B+ tree ensures that all leaf nodes are at the same height. Leaf nodes are linked in a link list. Therefore, you can access elements both sequentially and randomly in a b+ tree.

Construction

Question;
20,11,14,25,30,12,22,23,24
Answer:



SQL

Create Index

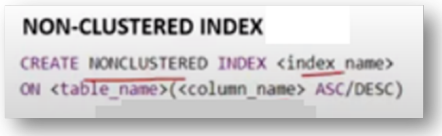
Syntax:
CREATE INDEX index_name
ON table_name (column1, column2, ...);

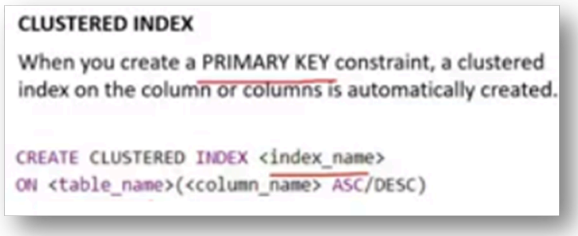
CREATE UNIQUE INDEX

Duplicate values are not allowed.
Syntax:
CREATE UNIQUE INDEX index_name
ON table_name (column1, column2, ...);

DROP INDEX

The DROP INDEX statement is used to delete an index in a table.
ALTER TABLE table_name DROP INDEX index_name;





Extra:
If table

ID	Dept
1	CS
2	CS
3	IT

And Index is created on Dept, index stores something like:

CS → row1, row2
IT → row3

Module 4.3

Query Processing and Optimization

Query Processing

Query processing is the process of converting an SQL query into an efficient execution plan and then executing it.

Steps in Query Processing

1. Parsing and Translation
2. Optimization
3. Evaluation

Parsing and Translation

Scanner: Identifies language tokens like SQL keywords, table names and attribute names.

Parser: Checks syntax of the query according to grammar rules.

Validation: Checks whether relation names and attribute names exist and are semantically correct according to schema.

Two internal representations of query:

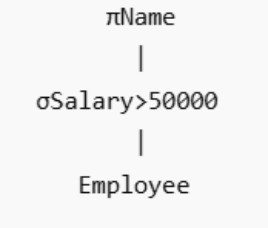
1. Query Tree
2. Query Graph

Query Tree

A hierarchical representation of a relational algebra query.
Used internally by most RDBMS.

- Leaf nodes: Relations/tables.
- Internal nodes: Operations like SELECT, PROJECT, JOIN.
- Edges: Show data flow.
- Execution: Bottom-up.

Example:
πName(σSalary>50000(Employee))



First selection happens, then projection.

Query Graph
A non hierarchical representation corresponding to relational calculus.

- Nodes: Relations and variables involved in the query.
- Edges: Join conditions/predicates.
- Does not specify execution order.

Steps of Query Processing

Lexical Analysis

Breaks query into tokens and removes spaces/comments. It is also called Syntactic Analysis.

Example:
SELECT emp_name FROM employee WHERE salary>10000;

Tokens:
SELECT, emp_name, FROM, employee, WHERE, salary, >, 10000

Semantic Analysis

Checks:
Whether tables exist.
Whether columns exist.
Whether compared data types are compatible.

Example:
salary and 10000 must be comparable datatypes.

Translation to Relational Algebra

SQL queries are converted into relational algebra expressions because they are easier for optimizers.

Example:
SELECT emp_name FROM Employee WHERE salary>10000;

Relational algebra:
πemp_name(σsalary>10000(Employee))

Evaluation

After translation, DBMS chooses algorithms to execute each relational algebra operation.

Query Evaluation Plan

A detailed execution strategy specifying:

1. Algorithms to use
2. Indexes to use
3. Access paths

Example: to compare and find the maximum salary, use salary index instead of full table scan.

Query Optimization

Choosing the most efficient execution strategy for a query.

1. Query code generator: Generates executable code.
2. Runtime database processor: Executes generated code.

Example

- $\sigma_{salary<75000}(\pi_{salary}(Employee))$
- $\pi_{salary}(\sigma_{salary<75000}(Employee))$

A relational algebra expression might have equivalent expressions.

Possible evaluation plans:

1. Can use an index on salary to find instructors with salary < 75000,
2. Can perform complete relation scan and discard instructors with salary 75000

Optimization Types

1. Heuristic Optimization (Rule based)
   Uses transformation rules.
2. Cost Based Optimization
   Chooses plan with minimum estimated cost.

Nested Query

Example

```
SELECT Ename FROM Emp WHERE DeptID IN (SELECT DeptID FROM Dept WHERE
Dname='Sales');
```

Join Query

```
SELECT Ename
FROM Emp e JOIN Dept d
ON e.Did=d.Did
WHERE Dname='Sales';
```

Query Blocks
A query block is the basic SQL unit translated into relational algebra and optimized.

- Contains: SELECT-FROM-WHERE, GROUP BY, HAVING.
- Nested queries create separate query blocks.
- Aggregate operators must be included in the extended algebra. Example: algebra like COUNT, MAX, etc. should be included like $\mathcal{F}$MAX Salary(Employee)

SELECT Operation

Selection retrieves tuples satisfying conditions.

Notation:
σcondition(Relation)

Examples:
σSSN='123456789'(EMPLOYEE)
σDNO=5(EMPLOYEE)

Algorithms for SELECT

S1 Linear Search

Checks every record. Slow but works always.

Example: Search employee with salary>50000 by scanning whole table.

S2 Binary Search

Used when: File is ordered on key attribute. Faster than linear search.

Example: Search RollNo=15 in sorted file.

S3 Primary Index Search

Uses primary index or hash key to retrieve one record.

Example: Search employee using primary key EmployeeID.

S4 Primary Index for Multiple Records

Used for: >, >=, <, <= on ordered key.

Example: Salary > 50000.

Find first matching record then continue sequentially.

S5 Clustering Index

Used on non key attribute with clustering index.

Example: Dept='Sales'.

Retrieve all grouped Sales records.

S6 Secondary B+ Tree Index

Works for equality conditions and range queries. More efficient than S4.

Example: Salary>=40000.

S7 Conjunctive Selection

For AND conditions. Use index for one condition then test remaining conditions. Works like multiindex.

Example: SELECT * FROM Employee WHERE Dept='Sales' AND Salary > 50000;

S8 Composite Index

Uses combined index on multiple attributes.

Example:
Composite index:
(Dept, Salary)

Directly searches:
(Dept='Sales', Salary=60000)

S9 Intersection of Record Pointers

Uses multiple secondary indexes. Use one index for dept, one for salary, then have their intersection.

Example:
Dept='Sales' → {1,2,4}
Salary>50000 → {1,3,4}

Intersection:
{1,4}

S1 – Linear Search, S2 – Binary Search, S3 – Primary Index for one record, S4 – Primary Index for multiple records, S5 – Clustering Index, S6 – B+ Tree Index, S7 – Conjunctive Selection, S8 – Composite Index, S9 – Intersection of Record Pointers.

—----------

PROJECT Operation

PROJECT retrieves only specified columns and removes duplicates.

Notation:
πattribute_list(Relation)

Example:
πName,Salary(Employee)

Algorithm for PROJECT

1. Scan relation.
2. Extract required attributes.
3. Remove duplicates.

```
PROJECT(R, attributes)

1. Create empty relation Result
2. For each tuple t in relation R
3.     Extract required attributes from t
4.     Form new tuple t'
5.     Add t' to Result
6. Remove duplicate tuples from Result
7. Return Result
```

PROJECT with Duplicate Elimination

1. Read each tuple from relation R.
2. Extract only required attributes and store in temporary relation T′.
3. If projected attributes contain a key:
   directly copy T′ to T because duplicates cannot exist.
4. Otherwise:
   sort tuples in T′.
5. Compare adjacent tuples.
6. If tuples are same:
   skip duplicate tuple.
7. Store unique tuples in final relation T.

UNION (T ← R ∪ S)

1. Sort both R and S on same attributes.
2. Compare tuples from both relations one by one.
3. If tuple in R is smaller:
   output R tuple.
4. If tuple in S is smaller:
   output S tuple.
5. If tuples are equal:
   output only once and skip duplicate.
6. Add remaining tuples after one relation finishes.

INTERSECTION (T ← R ∩ S)

1. Sort both R and S on same attributes.
2. Compare tuples from both relations.
3. If R tuple is smaller:
   move in R.
4. If S tuple is smaller:
   move in S.
5. If tuples are equal:
   output tuple because it is common.
6. Continue until one relation ends.

DIFFERENCE (T ← R − S)

1. Sort both R and S on same attributes.
2. Compare tuples from both relations.
3. If R tuple is smaller:
   output it because no matching tuple exists in S.

4. If S tuple is smaller:
   move in S.
5. If tuples are equal:
   skip tuple because it exists in S.
6. Add remaining tuples of R after S finishes.

Pipelining

Output of one operation is directly passed to the next operation without temporary storage.

Also called stream based processing.

Example:
Selection result directly fed into JOIN.

Advantage:
1. Less disk I/O.
2. Faster execution.

Heuristic Query Optimization

Uses rules to reduce intermediate results.

Main heuristic:
Perform SELECT and PROJECT before the JOIN or other binary operations to reduce the size of any intermediate results.

Why?
Reduces tuples and attributes before expensive operations like joins.

Example:

Rule 1:
Replace Cartesian Product + Selection with JOIN.

Instead of:
$\sigma_{A.id=B.id}(A \times B)$

Use:
$A \bowtie B$

Rule 2:
Push SELECT downward.

Example:
σDept='Sales'(Employee ⋈ Department)

Apply selection before join.

Heuristic optimization of query trees

The same query could correspond to many different relational algebra expressions — and hence many different query trees.

The task of heuristic optimization of query trees is to find a final query tree that is efficient to execute

Transformation Rules

1. Cascade of SELECT
   σc1 AND c2(R) = σc1(σc2(R))
2. Commutativity of SELECT
   σc1(σc2(R)) = σc2(σc1(R))
3. Cascade of PROJECT
   Only the last projection matters.

   πList1 (πList2 (...(πListn(R))...)) = πList1(R)

4. Commuting SELECT with PROJECT
   If selection attributes exist in projection list:
   πA(σc(R)) = σc(πA(R))
5. Commutativity of JOIN
   R ⋈ S = S ⋈ R

   Smaller table -> put on left

6. Push SELECT through JOIN
   Apply conditions before join whenever possible.
7. Push PROJECT through JOIN
   Project unnecessary attributes early.
8. Set Operations
   UNION and INTERSECTION are commutative.
   Difference is not.
9. Associativity
   (R ⋈ S) ⋈ T = R ⋈ (S ⋈ T)
10. SELECT with Set Operations
    Selection distributes over UNION, INTERSECTION and DIFFERENCE.
    σDept='Sales'(R ∪ S) = σDept='Sales'(R) ∪ σDept='Sales'(S)
    Push selection down: first select then apply set operations

11. PROJECT with UNION
    Projection distributes over UNION.
    $\pi Name(R \cup S) = \pi Name(R) \cup \pi Name(S)$
12. Convert Cartesian Product + Selection into JOIN
    $\sigma c(R \times S) = R \bowtie c\ S$

Heuristic Optimization Algorithm

1. Break conjunctive SELECT conditions (Select conditions with multiple ANDs) into cascades.
2. Push SELECT downward to apply selections early before expensive operations like JOIN.
3. Rearrange JOINs using associativity.
4. Replace Cartesian product + SELECT with JOIN.
5. Push PROJECT downward to remove unnecessary attributes early.
6. Combine operations that can be executed by a single algorithm.

Query Execution Plans

Materialized Evaluation: Intermediate results of queries are stored temporarily.

Pipelined Evaluation: Results are directly passed to the next operation.

Module 5.1

Transaction Processing Concepts

Transaction

A transaction is a sequence of operations used to perform a logical unit of work.

Example:
Transfer ₹50 from account A to B.

1. read(A)
2. A=A−50
3. write(A)
4. read(B)
5. B=B+50
6. write(B)

A transaction accesses and possibly updates database items.

Main issues:

1. System failures/hardware crashes
2. Concurrent execution of transactions.

ACID Properties

Atomicity

Either all operations execute or none execute.

Example:
If the system crashes after deducting ₹50 from A but before adding to B, the database becomes inconsistent.
DBMS must undo partial changes.

Consistency

Transactions must preserve database consistency.

Example:
Before transfer:
A+B=300

After transfer:
A+B must still remain 300.

Isolation

Concurrent transactions should not interfere with each other. Isolation is ensured by running transactions serially.

Example:
If another transaction reads the database after A reduced but before B increased, it sees incorrect total balance.

Durability

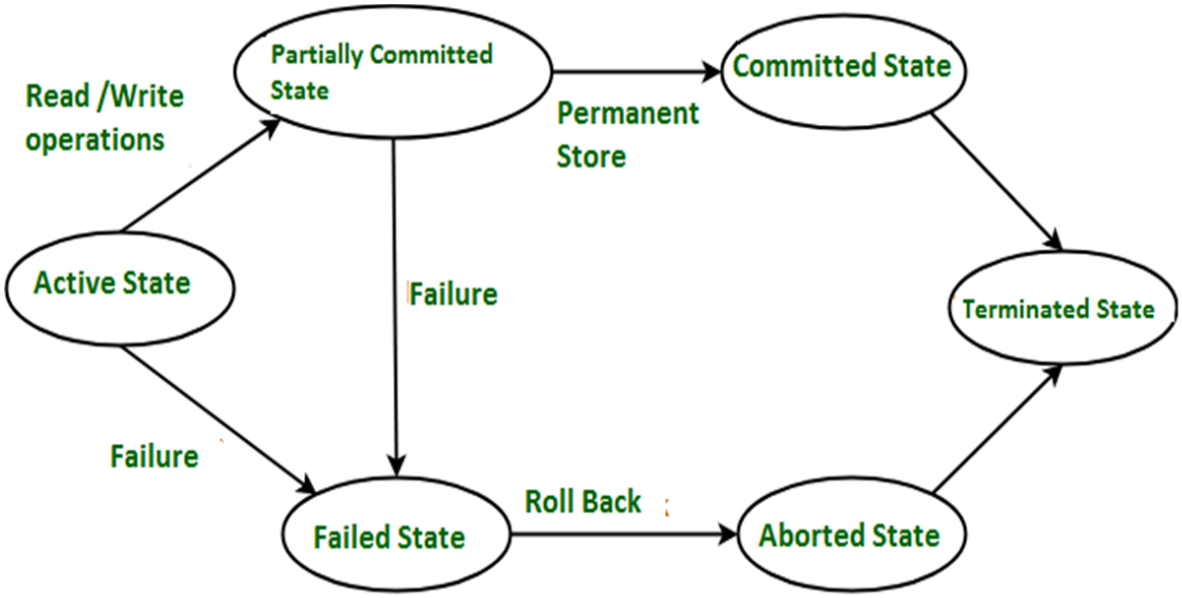
Once a transaction commits successfully, changes must persist even after crashes.

Transaction States

- Active
  Transaction is executing.
- Partially Committed
  Final statement executed. All statements executed, but changes may not yet be permanently saved to disk. (not yet committed)
- Failed
  Transaction cannot continue.
- Aborted
  Changes rolled back.
- Committed
  Transaction successfully completed and changes are permanently stored.

After abort, we have two options:

1. Restart transaction: Can only be done if there is no internal logical error
2. Kill transaction



Transaction States in DBMS

Implementation of Atomicity and Durability

The recovery management component of DBMS ensures atomicity and durability.

Shadow Database Scheme

All updates are first made on a shadow copy of database instead of original database.

A pointer called db_pointer always points to the current consistent database copy.

Transaction Commit Process

1. Create shadow copy.
2. Perform all updates on shadow copy.
3. After transaction reaches partially committed state and all pages are written to disk:
   change db_pointer to shadow copy.
4. Shadow copy now becomes an actual database.

Old original copy can then be deleted or overwritten later.

Transaction Failure
If transaction fails before commit:

1. Old database copy remains unchanged.
2. Shadow copy deleted.
3. Database stays consistent.

Disadvantages of Shadow Database Scheme

1. Assumes only one active transaction at a time.
2. Assumes disk never fails.
3. Very inefficient for large databases because complete copying required.
4. Poor concurrency support.
5. Shadow paging reduces copying overhead but is still impractical for large DBMS. Shadow Paging = Instead of copying the whole database, only modified pages are copied.

Concurrent Executions

Multiple transactions execute simultaneously.

Advantages:

1. Better CPU and disk utilization.
2. Reduced waiting time.
3. Higher throughput.

Concurrency Control

Mechanisms ensuring concurrent schedules remain correct and consistent.

Schedule
A schedule is the chronological execution order of transaction instructions.

Rules:

1. Must contain all instructions.
2. Must preserve instruction order inside each transaction.

Serial Schedule

Transactions execute completely one after another without overlapping.

Example:
```
 T1:
 read(A)
 write(A)
```

T2:
 read(B)
 write(B)

Serial schedule:
 T1 executes fully first,
 then T2 starts.

Non Serial Schedule
 Instructions of multiple transactions are interleaved.

Example:
 read(A) by T1
 read(B) by T2
 write(A) by T1
 write(B) by T2

Transactions overlap during execution.

Serializable Schedule
 A non serial schedule whose final result is equivalent to some serial schedule.

Example:

T1:
 read(A)
 A=A+10
 write(A)

T2:
 read(B)
 B=B+20
 write(B)

Schedule:
 read(A) T1
 read(B) T2
 write(A) T1
 write(B) T2

Even though interleaved,
 result is same as executing T1 then T2 serially,
 so schedule is serializable.

Types:

1. Conflict serializability.
2. View serializability.

Conflicting Instructions

Two instructions conflict if:

1. They belong to different transactions.
2. They access the same data item.
3. At least one performs write.

Cases:
read-read → no conflict
read-write → conflict
write-read → conflict
write-write → conflict

Conflict Serializability

A schedule is conflict serializable if it can be converted into a serial schedule by swapping non-conflicting instructions. Final result becomes a serial schedule.

Conflict Equivalent
Two schedules are conflict equivalent if one can be transformed into another by swapping non-conflicting operations.

Example:
r1(A) r2(B) w1(A) w2(B)

Since operations access different items, swaps possible. Possible swap: r1(A) w1(A) r2(B) w2(B)

View Serializability

Two schedules are view equivalent if:

1. Same transactions read initial values.
2. Same transaction performs final write.
3. Same transaction supplies values read by others.

A schedule is view serializable if its reads and final writes produce the same result as some serial schedule.

Serial schedule:
 T1: write(A=10)
 T2: read(A=10)

Concurrent schedule:
 write(A=10) by T1
 read(A) by T2

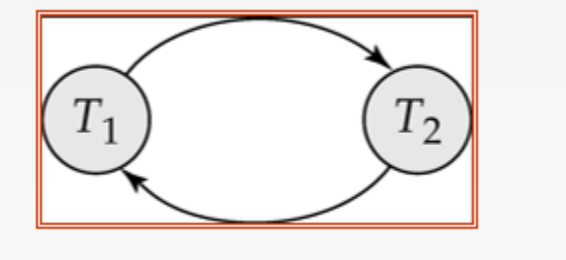
T2 reads same value and final result is same, so concurrent schedule is view serializable. Every conflict serializable (rearrange to make serial schedule) schedule is also view serializable.

Blind Write: A write operation whose value is never read later.

Schedules that are view serializable but not conflict serializable usually contain blind writes.

Testing Serializability

Precedence Graph / Serialization Graph



If the precedence graph has no cycle, then it is conflict serializable.
Possibility of schedule:

- T1->T2->T3,
- T1->T3->T2,
- T2->T1->T3,
- T2->T3->T1,
- T3->T1->T2,
- T3->T2->T1

Finding Equivalent Serial Order
Transaction with zero incoming edges executes first.

Recoverable Schedule

If Tj reads data written by Ti, then Ti must commit before Tj commits. Otherwise rollback problems occur.

Irrecoverable Schedule

Tj commits before Ti commits after reading Ti’s uncommitted data. If Ti aborts later, recovery impossible.

Cascading Rollback

Failure of one transaction forces rollback of dependent transactions.
Example:
T2 reads uncommitted value written by T1.
If T1 aborts, T2 must also abort.

Cascadeless Schedule

Transaction can read data only after writer transaction commits. Prevents cascading rollback. Every cascadeless schedule is recoverable.

Strict Schedule

A transaction cannot read or write a data item until previous writer commits or aborts. Allows only committed reads and writes. Strict schedule is stricter than cascadeless schedule.

Module 5.2,5.3

Concurrency Control

Concurrency control ensures simultaneous execution of multiple transactions without causing data inconsistency.

Why Needed?

Executing only one transaction at a time increases waiting time and reduces throughput.

Concurrency control:

- Allows concurrent execution.
- Maintains consistency.
- Prevents incorrect schedules.

Concurrency Control vs Serializability Testing

Concurrency control protocols:

- Concurrency control protocols work while transactions are executing, they don't let a bad schedule happen in the first place, rather than checking afterwards
- Suppose T1 and T2 both want to update the same bank balance. Without any protocol, they might interleave in a way that causes a lost update. A protocol will simply block T2 from acquiring a lock on that data until T1 is done, so the problematic interleaving never even occurs.
- Formally,CCP prevent non-serializable schedules during execution itself.

Serializability tests:

- Check whether a schedule is serializable after construction.
- Serializability tests look at a schedule that has already been built and then check if it is serializable or not.
- Suppose T1 and T2 have already executed and their operations are interleaved in some order. The test (like building a precedence graph) is applied after the fact to see if that interleaving was valid or not. It does not stop anything during execution, it only judges the result.

Protocols ensure that our schedules are:

- Serializable.

- Recoverable.
- Preferably cascadeless. Cascadeless means that if a transaction fails and rolls back, it does not force other transactions to roll back along with it.

Weak Levels of Consistency

Some applications allow approximate results for better performance.

Example: Approximate bank statistics.

Tradeoff: Less consistency but higher performance.

Levels of Consistency in SQL-92

Serializable

- Highest consistency level.
- Transactions behave as if executed serially.

Repeatable Read

- Only committed data can be read.
- Repeated reads return same value.
- But new inserted rows may appear from an operation by another transaction.
- Basically, you read the same rows and they stay the **same**. But new rows can appear.

Read Committed

- Only committed data readable.
- Repeated reads may return different committed values.
- You can re-read the same row and get a **different** value if another transaction updated and committed it in between.

Read Uncommitted

- Even uncommitted data can be read.
- May cause dirty reads.

Transaction Definition in SQL

Transaction starts implicitly. The database starts a transaction automatically the moment you run your first SQL statement.

A transaction in SQL can end by commit and rollback.

- COMMIT: Makes changes permanent and starts new transaction.
- ROLLBACK: Aborts current transaction.

Auto Commit

Most DBMS automatically commit every successful SQL statement. Can be disabled.

Example: connection.setAutoCommit(false)

Concurrency Control Problems

- Transactions mainly perform: Read, Write.
- concurrent execution increases throughput, but if done without any control it can create inconsistencies.
- Dirty Read Problem (Write-Read Conflict)
  Say T1 wrote something in A that T2 read but T1 failed aage jaake and rolled back. Now t2 did not rollback and has read the wrong value of A. Inconsistency.
- Loss Update Problem
  when two or more transactions modify the same data.

Concurrency Control Protocols

- Two Phase Locking Protocol (2PL)
- Timestamp Ordering Protocol
- Multi Version Concurrency Control (MVCC)
- Validation Based Protocol

Two Phase Locking Protocol (2PL)

- 2PL guarantees that the schedule produced will be equivalent to some serial schedule
- It does this by controlling when transactions can acquire and release locks.
- A lock is a mechanism that restricts other transactions from accessing a data item while one transaction is using it.

Two phases:

Growing Phase

- Transaction can acquire locks.
- Phase continues till said transaction cannot acquire more locks
- Can add as many locks but cannot release them

Shrinking Phase

- Transaction can release locks.
- Cannot acquire new locks.

Rule: Once first lock released, no new lock can be acquired.

Purpose: Prevent conflicting concurrent operations.

Timestamp Ordering Protocol

Transactions execute according to timestamps. Older transaction gets higher priority.

Timestamp may use:

- System clock
- Logical counter

Example:

- T1 timestamp = 007
- T2 timestamp = 009
- T1 executes first because it is older.

Important Terms

- TS(Ti): Timestamp of transaction Ti.
- R_TS(X): Largest timestamp of any transaction that read X.
- W_TS(X): Largest timestamp of any transaction that wrote X.

Read Rule

When Ti performs Read(X):

- If W_TS(X) > TS(Ti): reject operation because younger transaction already wrote X. Allowing Ti to read that would be reading stale stale data from the past, which would mess up the order. So Ti is rejected and rolled back.
- Else: allow read.

Write Rule

When Ti performs Write(X):

- If TS(Ti) < R_TS(X): reject because younger transaction already read X.
- If TS(Ti) < W_TS(X): reject and rollback because younger transaction already wrote X.
- Else: allow write.

Multi Version Concurrency Control (MVCC)

Maintains multiple versions of same data item instead of locking. When a transaction modifies data, a new version is created instead of overwriting the old version.

Advantages

- Read operations do not block write operations
- Higher concurrency: since readers don't block writers and writers don't block readers, more transactions can run simultaneously.
- Reduces deadlocks: deadlocks usually happen when transactions are waiting for each other's locks, and since MVCC readers don't take locks at all, that waiting is largely eliminated.

Example:

- Old balance = 1000.
- T1 updates to 1200.
- Database temporarily stores: Version 1 = 1000, Version 2 = 1200.
- Other transactions can still read older version.

MVCC Features

1. Multiple Versions: Different versions of same data exist simultaneously.
2. Non-Blocking Reads: Readers continue without waiting for writers.
3. Timestamps/Transaction IDs: Each version tagged using timestamp or transaction ID.
4. Garbage Collection: Old unused versions removed later.
5. Conflict Resolution: Usually first committed transaction succeeds. Other conflicting transaction rolled back.

Validation Based Protocol (Optimistic Concurrency Control)

Assumes conflicts are rare. Transactions execute without locking initially.

Three Phases:

1. **Read phase:** the transaction reads from the database but all its writes go to a temporary local copy, not the actual database.
2. **Validation phase:** before committing, it checks if any other transaction that ran concurrently would cause a conflict, basically did anyone else touch the same data.
3. **Write phase**: if no conflict found, the local copy is written to the actual database. If conflict found, the whole transaction is discarded and restarted.

Timestamps Used

1. Start(Ti): Time transaction started.
2. Validation(Ti): Time validation started.
3. Finish(Ti): Time write phase completed.

The order in which transactions are considered to have executed is determined by when they entered the validation phase, not when they started or when they finished writing.

So if T1 validates at time 5 and T2 validates at time 7, the schedule is treated as if T1 ran before T2, regardless of when they actually started reading.

Advantages

- High concurrency.
- Fewer locks.
- Good when conflicts are rare.

Deadlock

Deadlock occurs when transactions wait forever for each other's locks.

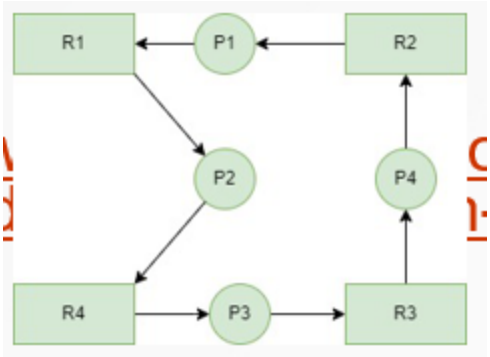
Example:

- T1 holds R1 and waits for R2.
- T2 holds R2 and waits for R1.
- Cycle formed → deadlock.

Deadlock Avoidance: Detects unsafe situations beforehand to avoid deadlock.

Deadlock Detection: DBMS checks whether waiting transactions form a cycle.

Wait For Graph



Graph used for deadlock detection.

Nodes: Processes/transactions. Edge: Ti → Tj means Ti waits for resource held by Tj.

Cycle present: deadlock exists.

Resource Allocation Graph vs Wait For Graph

- Resource Allocation Graph: Contains both processes and resources.
- Wait For Graph: Contains only processes. Resources removed.

Recovery System

Recovery from system failures like power cuts, crashes, or any unexpected shutdown that happens mid transaction.

Log Based Recovery

DBMS keeps log records on stable storage. Stable storage means storage that survives crashes, basically a hard disk as opposed to RAM which gets wiped on failure. Logs store all updates.

So every update is written to this log on disk before being applied to the actual database. If the system crashes, the log is still there and the DBMS can use it to figure out what happened and recover.

Log Records

Format: <Transaction, X, old_value, new_value> Example: <T1, A, 1000, 1200> means A changed from 1000 to 1200.

Purpose: Used for undo and redo during recovery.

Shadow Paging

Database divided into fixed size pages.

Uses:

- Shadow page table: Original stable database state.
- Current page table: Modified pages during transaction.

Working

- Transaction starts.
- Shadow table copied into current table.
- Updates applied only to new pages.
- Shadow table unchanged.

Commit Operation

- Write modified pages to disk.
- Save the current page table.
- Replace shadow page table pointer with current page table pointer.
- The current table now becomes a stable database.

Failure Before Commit

- Delete modified pages.
- Discard the current page table.
- Restore database using shadow page table.

Advantage: No redo operation required after commit.

Disadvantages

- Copying overhead.
- Poor concurrency support.
- Inefficient for very large databases.

Deferred Update (NO-UNDO/REDO)

- Updates are not written to disk until commit
- Stored in log/buffer first
- If crash before commit → just discard log, no undo needed
- If crash after commit → use log to redo
- Hence: NO-UNDO, REDO

Immediate Update (UNDO/NO-REDO)

- Updates written to disk immediately, even before commit
- If crash before commit → must undo changes
- If crash after commit → nothing to redo, already on disk
- Hence: UNDO, NO-REDO

LAB/OST

View:

```
CREATE VIEW rich_accounts AS
SELECT account_number, balance
FROM account
WHERE balance > 50000;

SELECT * FROM rich_accounts;
```

Trigger:

```
CREATE FUNCTION check_balance()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.balance < 0 THEN
    RAISE EXCEPTION 'Negative balance not allowed';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check
BEFORE INSERT OR UPDATE ON account
FOR EACH ROW
EXECUTE FUNCTION check_balance();
```

Procedure

```
CREATE PROCEDURE update_balance(acc INT, amt INT)
LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE account
  SET balance = balance + amt
  WHERE account_number = acc;
END;
$$;

CALL update_balance(101, 500);
```

Function

```
CREATE FUNCTION get_balance(acc INT)
RETURNS INT AS $$
DECLARE bal INT;
BEGIN
  SELECT balance INTO bal
  FROM account
  WHERE account_number = acc;

  RETURN bal;
END;
$$ LANGUAGE plpgsql;
```

Cursor:

```
DO $$
DECLARE rec RECORD;
BEGIN
  FOR rec IN SELECT * FROM account LOOP
    RAISE NOTICE '%', rec.balance;
  END LOOP;
END;
$$;

—-----------------------------------------------------------------------

DO $$
DECLARE
  rec RECORD;
  cur CURSOR FOR SELECT name, balance FROM account;
BEGIN
  OPEN cur;

  LOOP
    FETCH cur INTO rec;
    EXIT WHEN NOT FOUND;

    RAISE NOTICE '% %', rec.name, rec.balance;
  END LOOP;

  CLOSE cur;
END;
$$;
```

Tcl:

```
BEGIN;
UPDATE account SET balance = balance - 100 WHERE account_number = 1;
UPDATE account SET balance = balance + 100 WHERE account_number = 2;
COMMIT;

ROLLBACK;
```

DCL

```
GRANT SELECT ON account TO user1;
REVOKE SELECT ON account FROM user1;
```

INDEX:

```
CREATE INDEX idx_balance ON account(balance);
```

SCHEMA

- **branch** (branch_name, branch_city, assets)
- **customer** (ID, customer_name, customer_street, customer_city)
- **loan** (loan_number, branch_name, amount)
- **borrower** (ID, loan_number)
- **account** (account_number, branch_name, balance)
- **depositor** (ID, account_number)

Find customers who have accounts in all branches located in 'Mumbai'.

```
SELECT c.customer_name
FROM customer c
WHERE NOT EXISTS (
  SELECT b.branch_name
  FROM branch b
  WHERE b.branch_city = 'Mumbai'
  AND b.branch_name NOT IN (
    SELECT a.branch_name
    FROM depositor d
    JOIN account a ON d.account_number = a.account_number
    WHERE d.ID = c.ID
  )
);
```

Find customers who have taken a loan but do NOT have any account.

```
SELECT DISTINCT c.customer_name
FROM customer c
JOIN borrower b ON c.ID = b.ID
WHERE c.ID NOT IN (
  SELECT d.ID FROM depositor d
);
```

Find branch names where total loan amount exceeds 1,00,000.

```
SELECT branch_name, SUM(amount) AS total_loan
FROM loan
GROUP BY branch_name
HAVING SUM(amount) > 100000;
```

Find customers whose account balance is greater than average balance of all accounts.

```
SELECT DISTINCT c.customer_name
FROM customer c
JOIN depositor d ON c.ID = d.ID
JOIN account a ON d.account_number = a.account_number
WHERE a.balance > (
  SELECT AVG(balance) FROM account
);
```

Questions

Module 1

1. Lifecycle of data
2. DBMS architecture tiers
3. DML vs DDL

Module 2

1. Relational model constraints list and explain

Module 3

1. Classification of SQL Commands
2. Sql commands
3. What are views why do we have them
4. What are triggers why do we have them
5. What are cursors why do we have them
6. Explain commit rollback and savepoint
7. Differentiate bw functions and procedures

Module 4

1. Decomposition into 2nf 3nf
2. Why is normalization needed what is normal form
3. What are update anomalies in a relational database? Explain insertion, deletion, and updation anomaly with a suitable example for each.
4. What are the types of decomposition? What is the difference between them
5. Guidelines for Relational Database Design
6. Infer if F is a closure on G
7. Infer if FG is equivalent
8. Write and explain Inference Rules
9. Benefit of indexing and drawback
10. Attributes of indexing
11. Why are b+tree leaf nodes linked
12. Write and explain steps of Query Processing
13. Explain all stages for SELECT Operation
14. How does union, intersection, difference work internally? Algo
15. Write steps of heuristic optimization algorithm

Module 5

1. List and describe ACID properties
2. List and explain transaction states with diagram
3. Wdym Shadow Database Scheme. What disadvantages
4. difference between Concurrency Control vs Serializability Testing
5. Four Levels of Consistency in SQL-92
6. List the Concurrency Control Protocols

SKIP

Cost Based Query Optimization: Chooses plan with lowest estimated execution cost.

Cost Components

1. Disk access cost.
2. Storage cost.
3. Computation cost.
4. Memory usage cost.
5. Communication cost.

Catalog Information Used
r = number of tuples.
R = record size.
b = number of blocks.
bfr = blocking factor.

Index information:
x = index levels.
d = distinct attribute values.
sl = selectivity.
s = selection cardinality.

Selectivity
Fraction of tuples satisfying a condition.

Example:
100 matching tuples out of 1000.

sl = 100/1000 = 0.1

Selection cardinality:
s = sl × r

Join Selectivity
Fraction of tuples satisfying join condition.

$$js = \frac{|R \bowtie S|}{|R|\times|S|}$$

Join Result Size

$$|R \bowtie S| = js \times |R| \times |S|$$

Join Algorithms

J1 Nested Loop Join
Compare every tuple of R with every tuple of S.

Slow for large relations.

J2 Single Loop Join
Uses index on join attribute.

Faster than nested loop.

J3 Sort Merge Join
Sort both relations then merge matching tuples.

Efficient for ordered relations.

Join Ordering
For n relations:
Need n−1 joins.

Many possible join orders exist.

Left Deep Tree
Right child is always a base relation.

Advantages:
Good for pipelining.
Can use indexes efficiently.

Oracle Query Optimization
Oracle supports:
Rule based optimization.
Cost based optimization.

Cost based optimization is preferred.

Oracle may also use hints provided by developers.

Semantic Query Optimization
Uses database constraints to optimize queries.

Example constraint:
Employee salary cannot exceed supervisor salary.

Query:

```
SELECT E.LNAME, M.LNAME
FROM EMPLOYEE E, EMPLOYEE M
```

```
WHERE E.SUPERSSN=M.SSN
AND E.SALARY>M.SALARY;
```

Optimizer realizes result will always be empty because of constraint, so query need not execute.